



IKT300

Software Architecture and Design

Arne Wiklund



Arne Wiklund

Office: A3-067

arnew@uia.no

Don't message me in Canvas
if you want a response 😊

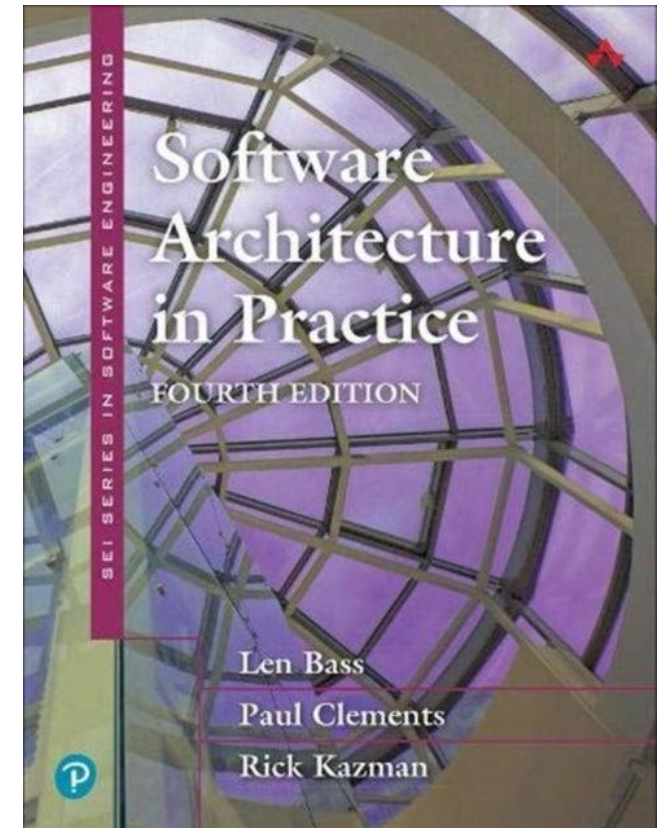
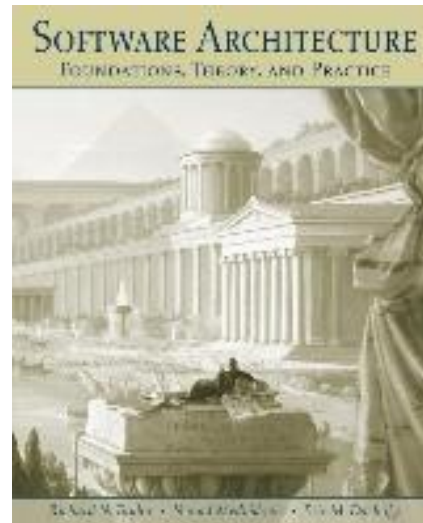
- Lectures:
 - Tuesdays 13:15-15:00
 - Friday 12:15 – 13:00
- Lab:
 - Tuesdays 15:15-17:00
 - A3-021/022
 - Can we do something with this?
- Come to lab if you want help

Objectives

- Good knowledge of the software development process and its phases.
- Understand the relationship between requirements, architecture and tradeoffs.
- Know methods and techniques for designing software systems.
- Express software architecture and design of a system using UML.

Curriculum

- Software Architectre in practice 4th edition – chapter 1-15
- Slides and presentaions in class
- Handouts
- Software Architecture, Foundation, theory and practice



Evaluation

- Written exam (70%) multiple choice
- Project (30%)
 - Oral examination
 - The project can not be retaken outside ordinary enrollment (hence, no “konte” in the spring)

Projects and hand-ins

- To take the exam:
 - There will be 4 hand-outs that must be read, summarized, criticized and per-reviewed.
 - You will need to do some software projects hand-ins
- The last handin is a bigger project, that will be:
 - a report
 - working software
 - it will be graded (30% of total grade)

Software to be used

- Enterprise Architect (EA) from Sparx systems
 - If you have Mac or Linux and don't get EA running, you can get a Windows virtual machine.
- Visual studio (VS) and C#
- Use this week to get started with EA and VS/C#

Design Concepts

- Design and architecture is everywhere:
 - International space station
 - Weekend party

Quiz 1

Can you distinguish the following definitions:

Design:	A: skillful/artful contrivance applying scientific and mathematical principles
Engineering:	B: use of skill, taste, and imagination in the production of aesthetic objects
Craft:	C: deliberate, purposive planning
Art:	D: skilled occupation

Quiz 1

Can you distinguish the following definitions:

Design: C	A: skillful/artful contrivance applying scientific and mathematical principles
Engineering: A	B: use of skill, taste, and imagination in the production of aesthetic objects
Craft: D	C: deliberate, purposive planning
Art: B	D: skilled occupation

Quiz 2

- Which of the following are differences between software design (as a process) and programming?
 - ☐ Need for abstraction
 - ☐ Scale
 - ☐ Use of data type
 - ☐ Emphasis on Non-functional requirements (NFR)
 - ☐ All of the above

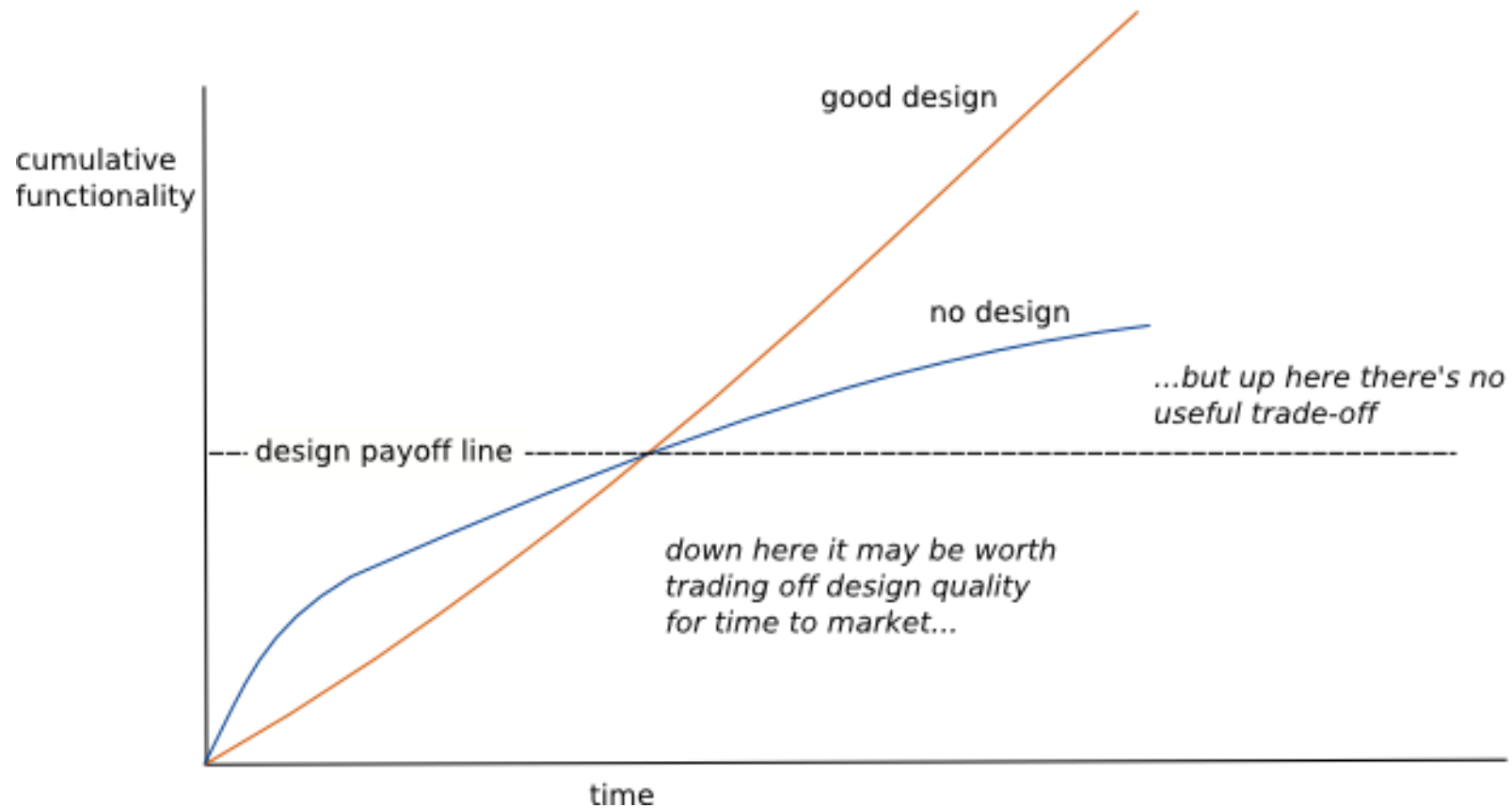
Quiz 2

- Which of the following are differences between software design (as a process) and programming?
 - ☐ Need for abstraction
 - ☒ Scale
 - ☐ Use of data type
 - ☒ Emphasis on Non-functional requirements (NFR)
 - ☐ All of the above

Software design

- The process of building a software system while satisfying functional requirements and not violating non-functional constraints.

Why software design



source: <https://martinfowler.com/bliki/DesignStaminaHypothesis.html>

Technical debt

Reckless

Prudent

«We don't have time for design»

«We must ship now and deal with the consequences later»

«What is layering?»

«Now we know how we should have done it»

Deliberate
Inadvertent

Design rationale

«Design decisions are explicit choices of how to trade off non-functional aspects of a design.»

speed vs size

low cost vs high quality

security vs light weight

Design representation

- Pseudo code
- Structured programming
- Flowcharts
- Diagram
- Decision tables

Conceptual integrity



Kenangan Palace in Kuala Kangsar, Perak, Malaysia

Arne Wiklund

«The quality of the system where all the concepts and relationship with each other are applied in a consistent way throughout the system»

Conceptual integrity

«there is seldom so much perfection in works composed of many separate parts, upon which different heads had been employed, as in those completed by a single master» - Descartes

A camel is a horse designed by committee

An individual makes the case for a product (horse). It is introduced to few people who realize its potential. The stakeholders engage and define a process and committee (one representative from each stakeholder), with no one in lead. The committee develops a camel (the individual has probably moved on).

Per Wikipedia "Design by committee is a term referring to a style of design and its resultant output when a group of entities comes together to produce something (often the design of technological systems or standards), particularly in the presence of poor leadership. The defining characteristics of "design by committee" are needless complexity, internal inconsistency, logical flaws, banality, and the lack of a unifying vision."

Issues with design methods

- Conceptual integrity vs crowd development
- Tradeoff between maintainability vs time to market
 - Deliberate prudent technical depth

Other design issues

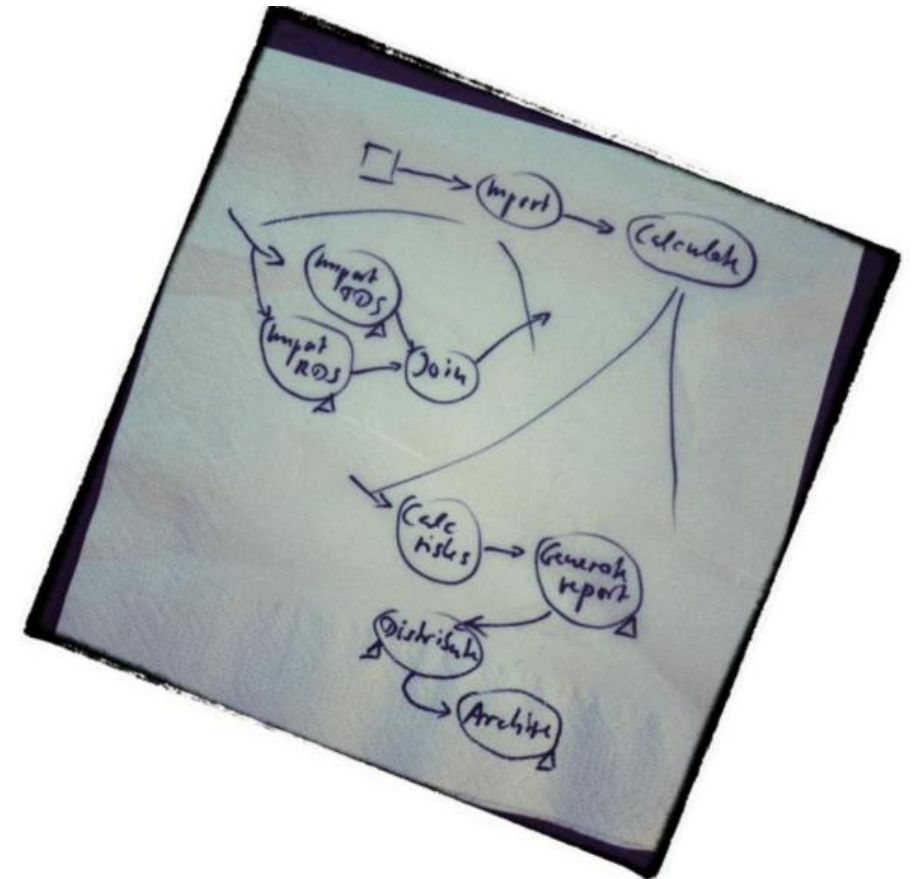
- Architectural vs detailed design
- Functional requirements vs NFR's
- Specification (what) vs design (how)
- Domain/application specificity

Design documentation



Archivers dream

Source: <http://www.montel.com>



The famous napkin dokumentation

Source: <http://geekswithblogs.net/theArchitectsNapkin>

Design documentation

- IEEE 1016 has a general good structure

TABLE OF CONTENTS

1. INTRODUCTION	2
1.1 Purpose	2
1.2 Scope	2
1.3 Overview	2
1.4 Reference Material	2
1.5 Definitions and Acronyms	2
2. SYSTEM OVERVIEW	2
3. SYSTEM ARCHITECTURE	2
3.1 Architectural Design	2
3.2 Decomposition Description	3
3.3 Design Rationale	3
4. DATA DESIGN	3
4.1 Data Description	3
4.2 Data Dictionary	3
5. COMPONENT DESIGN	3
6. HUMAN INTERFACE DESIGN	4
6.1 Overview of User Interface	4
6.2 Screen Images	4
6.3 Screen Objects and Actions	4
7. REQUIREMENTS MATRIX	4
8. APPENDICES	4

Design rationale

- «Design rationale captures the reasoning of the designer that led to the system as designed and the justifications of those decisions. » –IEEE 1016
- may include concerns, trade-offs, design options, decisions made, and arguments and justifications made to reach a decision.

Coupling and Cohesion

- Coupling – an indication of how much two components dependent on each other to function properly.
- Cohesion – an indication of how well to modules fit together. Usually achieved by assigning a single functionality to components.
- Low Coupling – high cohesion.

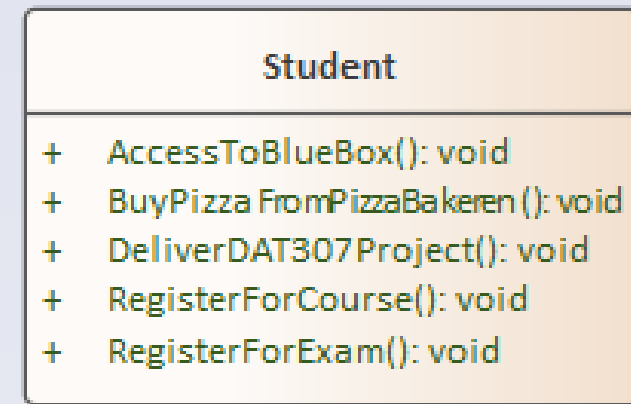
Example coupling

- Java package system reduce coupling by encapsulating names; must import package to get access to given functionality
- Interfaces reduce coupling
- OO class inheritance increase coupling between parent and child since the child depend upon (and must know about) the parent's implementation.

Favor composition over inheritance

Example cohesion

- What is the problem?



Design Principles (high level)

- Information hiding
- Abstraction
- Decomposition

Information hiding

- How do you encapsulate the capabilities of a particular module?
- Encapsulate the capabilities behind an abstract interface.
- Design for change. Gives you the freedom to change implementation without breaking users implementation.



Information hiding

- Suppress the internal details of a component
- Black boxing
- Prevents seepout of busi

```
class Test
{
    private Item[][] multiArray;

    public Item[] getMultiArray()
    {
        return multiArray;
    }

    public void setMultiArray (Item[][] ma)
    {
        multiArray = ma;
    }
}
```

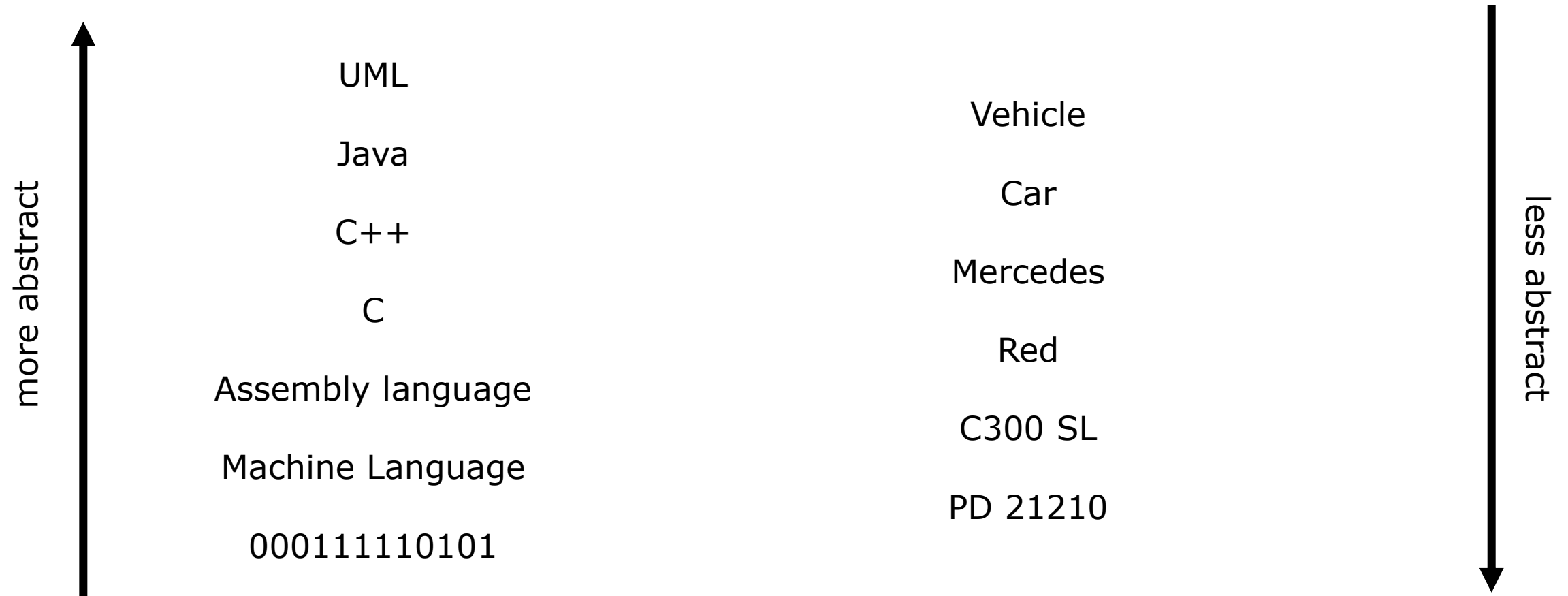
```
interface BusinessLogic
{
    1reference
    double CalculateTax(double amount);
}
```

```
public class TaxCalculation : BusinessLogic
{
    1reference
    double BusinessLogic.CalculateTax(double amount)
    {
        return amount * 1.25;
    }
}
```

Abstraction and refine

- Abstraction – a description of something that omits details that are not relevant to the purpose of the abstraction -> to generalize.
- Describing the what, not the how.
- Generalization in a class hierarchy
- Refinement – a detailed description that conforms to another (its abstraction).

Abstraction



Decomposition

- Decomposition – to subdivide a solution into separable components that are named and addressable
- Good strategy for big problems
 - Devide and conquer

What is Software Architecture?

The software architecture of a system is the set of structures needed to reason about the system, which comprise software elements, relations among them, and properties of both. – SAP

Alternative definitions

- [Lane 90]: ...structure and performance of software systems.... aspects include the division of functions among system modules, the means of communication between modules, and the representation of shared information.
- [Bhansali 92]: A topological organization of a set of parameterized modules,
- [Garlan 92]:global control structure; protocols for communication, synchronization, and data access; assignment of functionality to design elements; composition of design elements;and selection among design alternatives.
- [Perry 92]: processing elementsdata elements; ... connection elements rationale for the various choices...
- [Crispen 94]: ... a partitioning strategy and (b) a coordination strategy.
- [Clements 94-2]: (organization of ..) components, connections, constraints, and rationale.
- [Moriconi 94]: ...Component:... Interface: ...Connector, ...Configuration: ...Mapping....Architectural style:
- [Garlan 94]: ... high-level organization of computational elements and interactions between those elements.
- [FHayes-Roth 94]:....components described in terms of their behaviors and interfaces andinterconnections.
- [Abowd 95]: components protocols of interaction ... global system properties , ...life-cycle issues
- [Garlan 95]: The structure of components ... their interrelationships, and design and evolution guidelines over time.
- [ATA 96]: ...minimal set of rules governing the arrangement, interaction, and interdependence of the parts .

IEEE definitions

- *Architecture is the fundamental **organization** of a **system** embodied in its **components**, their **relationships** to each other, and to the **environment**, and the principles guiding its design and evolution. [IEEE 1471]*
- *A **system** is a collection of components organized to accomplish a specific function or set of functions. The term system encompasses individual applications, systems in the traditional sense, subsystems, systems of systems, product lines, product families, whole enterprises, and other aggregations of interest. A system exists to fulfill one or more **missions** in its environment. [IEEE 1471]*

IEEE definitions

- The **environment**, or context, determines the setting and circumstances of developmental, operational, political, and other influences upon that system. [IEEE 1471]
- The **environment**, or context, determines the setting and circumstances of developmental, operational, political, and other influences upon that system. [IEEE 1471]
- A **stakeholder** is an individual, team, or organization (or classes thereof) with interests in, or concerns relative to, a system. [IEEE 1471]

Philippe Kruchten

- An architecture is the **set of significant decisions** about the organization of a software system, the selection of **structural elements** and their interfaces by which the system is composed, together with their **behavior** as specified in the collaborations among those elements, the **composition** of these elements into progressively larger subsystems, and the **architectural style** that guides this organization -- these elements and their interfaces, their collaborations, and their composition. [Kruchten]

What is Software Architecture?

The software architecture of a system is the set of structures needed to reason about the system, which comprise software elements, relations among them, and properties of both. -SAP

Definition

- This definition stands in contrast to other definitions that talk about the system's "early" or "major" design decisions.
 - Many architectural decisions are made early, but not all are.
 - Many decisions are made early that are not architectural.
 - It's hard to look at a decision and tell whether or not it's "major."
- Structures, on the other hand, are fairly easy to identify in software, and they form a powerful tool for system design.

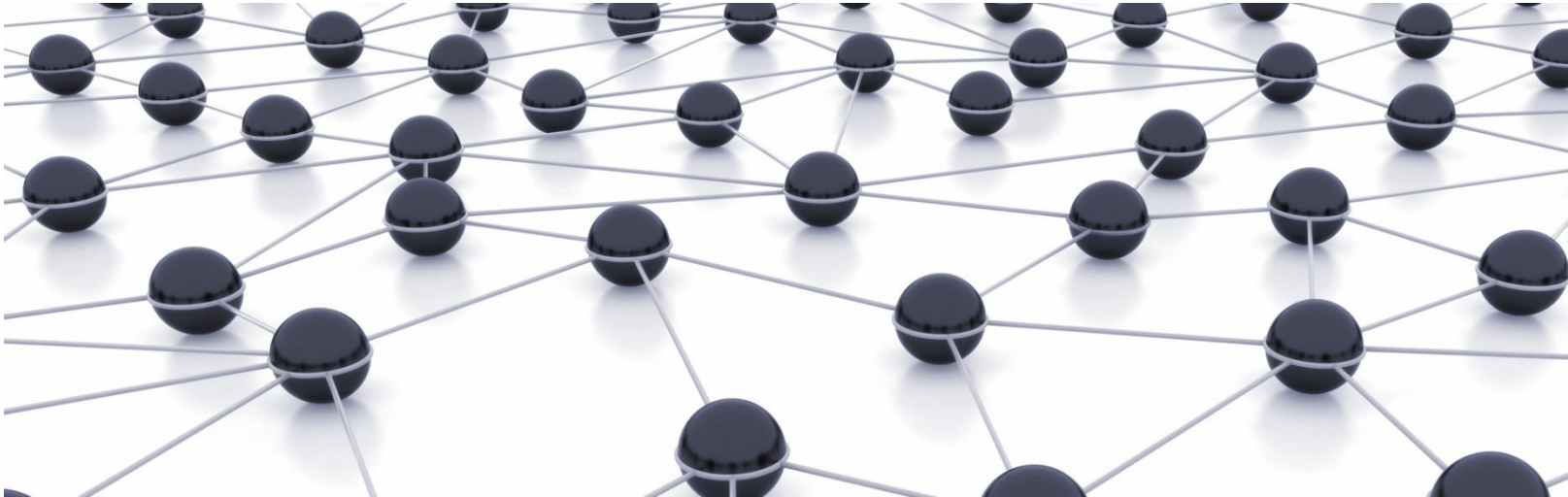
Is this good architecture?



Is this good architecture?



Is this good architecture?



What Makes a “Good” Architecture?

- There is no such thing as an inherently good or bad architecture.
- Architectures are either more or less fit for some purpose
- Architectures can be evaluated but only in the context of specific stated goals.
- There are, however, good rules of thumb.

Process “Rules of Thumb”

- The architecture should be the product of a single architect or a small group of architects with an identified technical leader.
 - This approach gives the architecture its conceptual integrity and technical consistency.
 - This recommendation holds for Agile and open source projects as well as “traditional” ones.
 - There should be a strong connection between the architect(s) and the development team.
- The architect (or architecture team) should base the architecture on a prioritized list of well-specified quality attribute requirements.
- The architecture should be documented using views. The views should address the concerns of the most important stakeholders in support of the project timeline.
- The architecture should be evaluated for its ability to deliver the system’s important quality attributes.
 - This should occur early in the life cycle and repeated as appropriate.
- The architecture should lend itself to incremental implementation,
 - Create a “skeletal” system in which the communication paths are exercised but which at first has minimal functionality.

Structural “Rules of Thumb”

- The architecture should feature well-defined modules whose functional responsibilities are assigned on the principles of information hiding and separation of concerns.
 - The information-hiding modules should encapsulate things likely to change
 - Each module should have a well-defined interface that encapsulates or “hides” the changeable aspects from other software
- Unless your requirements are unprecedented your quality attributes should be achieved using well-known architectural patterns and tactics specific to each attribute.
- The architecture should never depend on a particular version of a commercial product or tool. If it must, it should be structured so that changing to a different version is straightforward and inexpensive.
- Modules that produce data should be separate from modules that consume data.
 - This tends to increase modifiability
 - Changes are frequently confined to either the production or the consumption side of data.

Structural “Rules of Thumb”

- Don’t expect a one-to-one correspondence between modules and components.
- Every process should be written so that its assignment to a specific processor can be easily changed, perhaps even at runtime.
- The architecture should feature a small number of ways for components to interact.
 - The system should do the same things in the same way throughout.
 - This will aid in understandability, reduce development time, increase reliability, and enhance modifiability.
- The architecture should contain a specific (and small) set of resource contention areas, the resolution of which is clearly specified and maintained.

Architecture Is a Set of Software Structures

- A structure is a set of elements held together by a relation.
- Software systems are composed of many structures, and no single structure holds claim to being the architecture.
- There are three important categories of architectural structures.
 1. Module
 2. Component and Connector
 3. Allocation

Module Structures

- Some structures partition systems into implementation units, which we call modules.
- Modules are assigned specific computational responsibilities, and are the basis of work assignments for programming teams.
- In large projects, these elements (modules) are subdivided for assignment to sub-teams.

Component-and-connector Structures

- Other structures focus on the way the elements interact with each other at runtime to carry out the system's functions.
- We call runtime structures *component-and-connector (C&C) structures*.
- In our use, a component is always a runtime entity.
 - Suppose the system is to be built as a set of services.
 - The services, the infrastructure they interact with, and the synchronization and interaction relations among them form another kind of structure often used to describe a system.
 - These services are made up of (compiled from) the programs in the various implementation units – modules.

Allocation Structures

- Allocation structures describe the mapping from software structures to the system's environments
 - organizational
 - developmental
 - installation
 - Execution
- For example
 - Modules are assigned to teams to develop, and assigned to places in a file structure for implementation, integration, and testing.
 - Components are deployed onto hardware in order to execute.

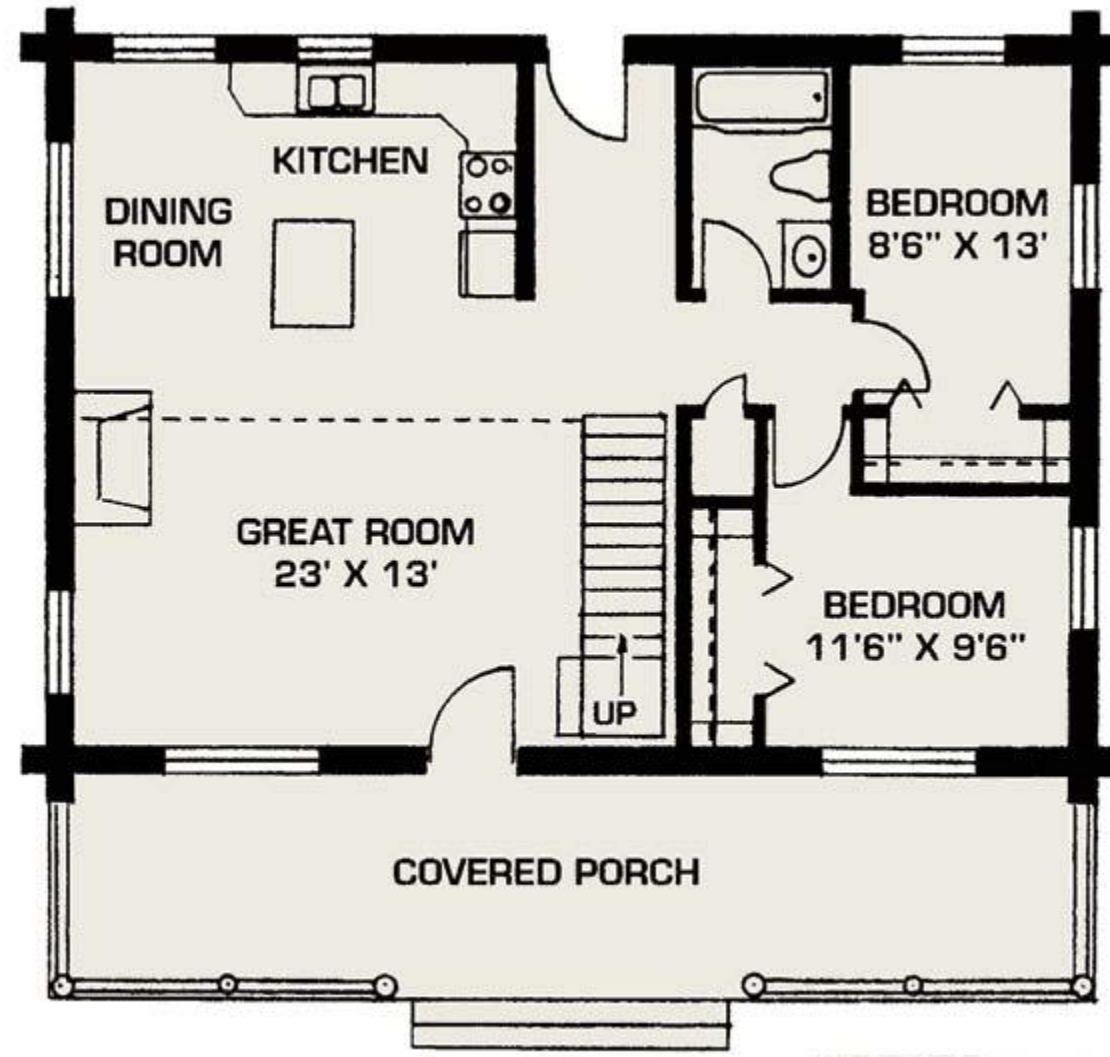
Which Structures are Architectural?

- A structure is architectural if it supports reasoning about the system and the system's properties.
- The reasoning should be about an attribute of the system that is important to some stakeholder.
- These include
 - functionality achieved by the system
 - the availability of the system in the face of faults
 - the difficulty of making specific changes to the system
 - the responsiveness of the system to user requests,
 - many others.
- Example: «The thermostat uses dial setting to control a heater to maintain a constant temperature in the room.»

Architecture is an Abstraction

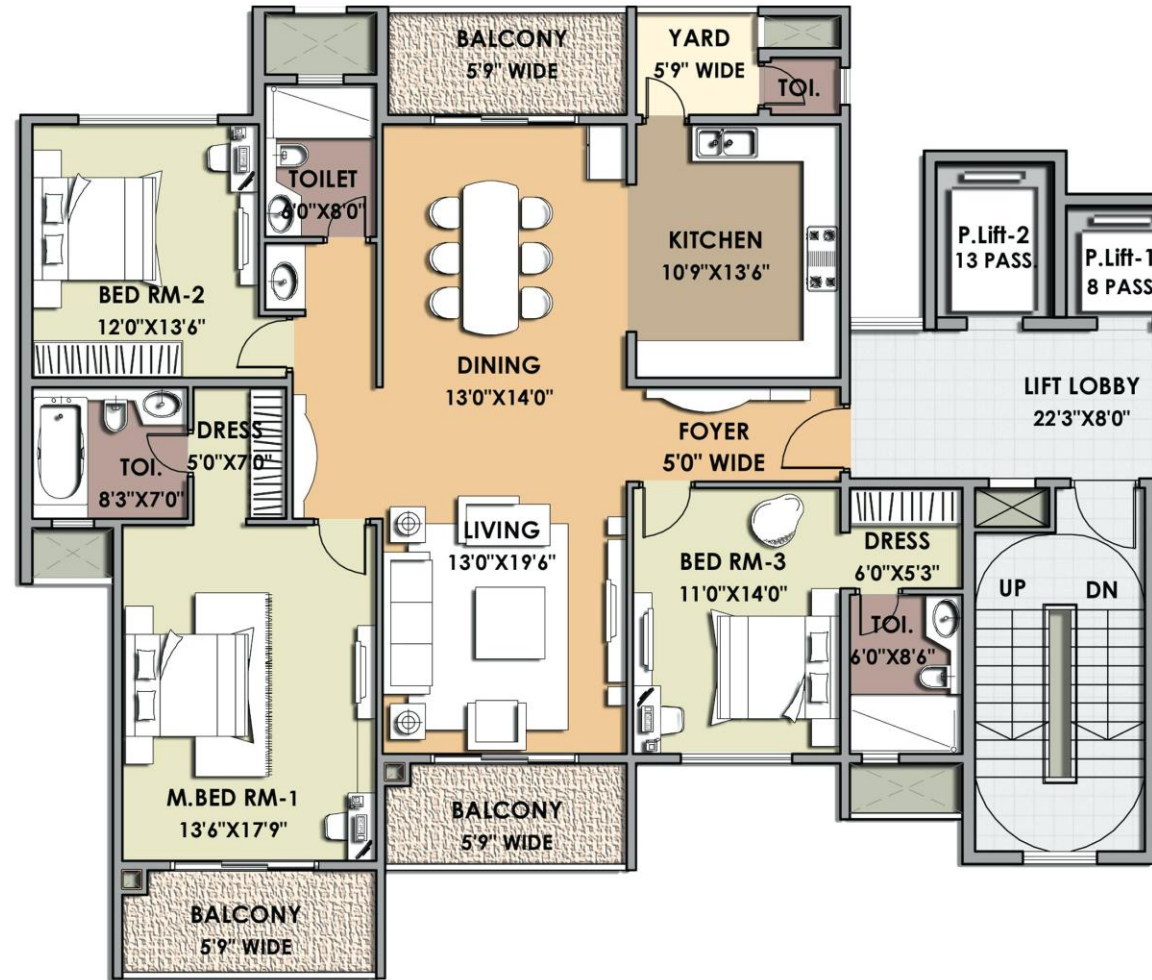
- An architecture comprises software elements and how the elements relate to each other.
 - An architecture specifically omits certain information about elements that is not useful for reasoning about the system.
 - It omits information that has no ramifications outside of a single element.
 - An architecture selects certain details and suppresses others.
 - Private details of elements—details having to do solely with internal implementation—are not architectural.
- The architectural abstraction lets us look at the system in terms of its elements:
 - how they are arranged,
 - how they interact,
 - how they are composed,
 - what their properties are that support our system reasoning,
 - and so forth.
- This abstraction is essential to taming the complexity of an architecture.
- We simply cannot, and do not want to, deal with all of the complexity all of the time.

Example #1



Stakeholders : 5 children family

Example #2



Stakeholders : 5 children family

Every System has a Software Architecture

- Every system comprises elements and relations among them to support some type of reasoning.
- But the architecture may not be known to anyone.
 - Perhaps all of the people who designed the system are long gone
 - Perhaps the documentation has vanished (or was never produced)
 - Perhaps the source code has been lost (or was never delivered)
- An architecture can exist independently of its description or specification.
- Documentation is critical.

Architecture Includes Behavior

- The behavior of each element is part of the architecture insofar as that behavior can be used to reason about the system.
- This behavior embodies how elements interact with each other, which is clearly part of the definition of architecture.
- Box-and-line drawings that are passed off as architectures are not architectures at all.
 - When looking at the names of the boxes a reader may well imagine the functionality and behavior of the corresponding elements.
 - But it relies on information that is not present – and could be wrong!
- This does not mean that the exact behavior and performance of every element must be documented in all circumstances.
 - Some aspects of behavior are fine-grained and below the architect's level of concern.
- To the extent that an element's behavior influences another element or influences the acceptability of the system as a whole, this behavior must be considered, and should be documented, as part of the software architecture.

Quiz

- Software architecture is an abstraction of a system that suppresses details of how software elements are internally.
- a) True
- b) False

Quiz

- Software architecture is an abstraction of a system that suppresses details of how software elements are internally.
- a) **True**
- b) False

Quiz

- The «externally visible properties» of a software architecture's elements refers to the assumptions that elements can make about one another.
- a) True
- b) False

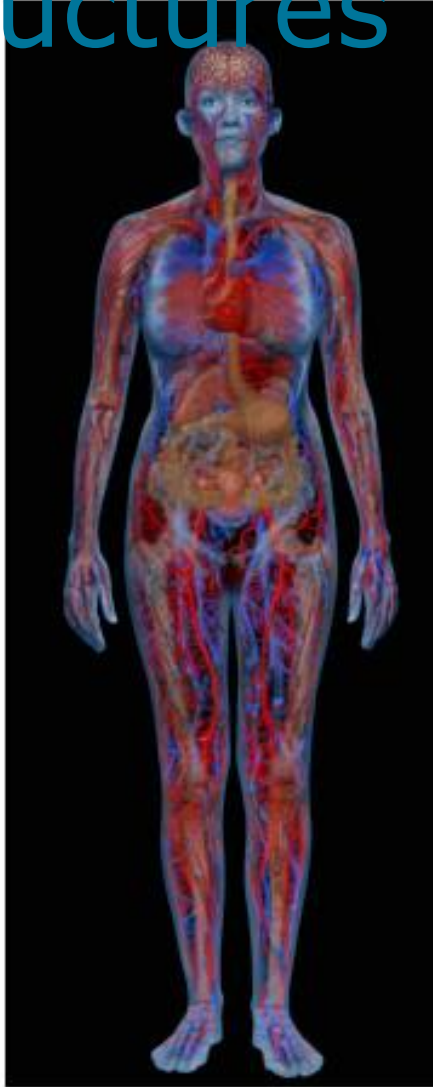
Quiz

- The «externally visible properties» of a software architecture's elements refers to the assumptions that elements can make about one another.
- a) **True**
- b) False

Physiological Structures

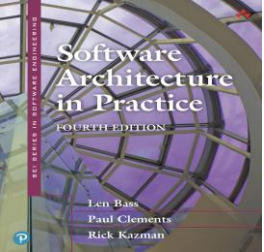
- The neurologist, the orthopedist, the hematologist, and the dermatologist all have different views of the structure of a human body.
- Ophthalmologists, cardiologists, and podiatrists concentrate on specific subsystems.
- The kinesiologist and psychiatrist are concerned with different aspects of the entire arrangement's behavior.
- Although these views are pictured differently and have different properties, all are inherently related, interconnected.
- Together they describe the architecture of the human body.
- So it is with software!

Physiological Structures



Structures and Views

- A *view* is a representation of a coherent set of architectural elements, as written by and read by system stakeholders.
 - A view consists of a representation of a set of elements and the relations among them.
- A *structure* is the set of elements itself, as they exist in software or hardware.
- In short, a view is a representation of a structure.
 - For example, a module *structure* is the set of the system's modules and their organization.
 - A module *view* is the representation of that structure, documented according to a template in a chosen notation, and used by some system stakeholders.
- Architects design structures. They document views of those structures.



Structures Provide Insight

- Structures play such an important role in our perspective on software architecture because of the analytical and engineering power they hold.
- Each structure provides a perspective for reasoning about some of the relevant quality attributes.
- For example:
 - The module structure, which embodies what modules use what other modules, is strongly tied to the ease with which a system can be extended or contracted.
 - The concurrency structure, which embodies parallelism within the system, is strongly tied to the ease with which a system can be made free of deadlock and performance bottlenecks.
 - The deployment structure is strongly tied to the achievement of performance, availability, and security goals.
 - And so forth.

Quiz

- Architecture is best described as a single, coherent structure.
- a) True
- b) False

Quiz

- Architecture is best described as a single, coherent structure.
- a) True
- b) False

Quiz

- A software system only has an architecture if one was explicitly designed and documented.
- a) True
- b) False

Quiz

- A software system only has an architecture if one was explicitly designed and documented.
- a) True
- b) **False**

Quiz

- «Box and line» diagrams are generally sufficient for describing a software architecture.
- a) True
- b) False

Quiz

- «Box and line» diagrams are generally sufficient for describing a software architecture.
- a) True
- b) **False**

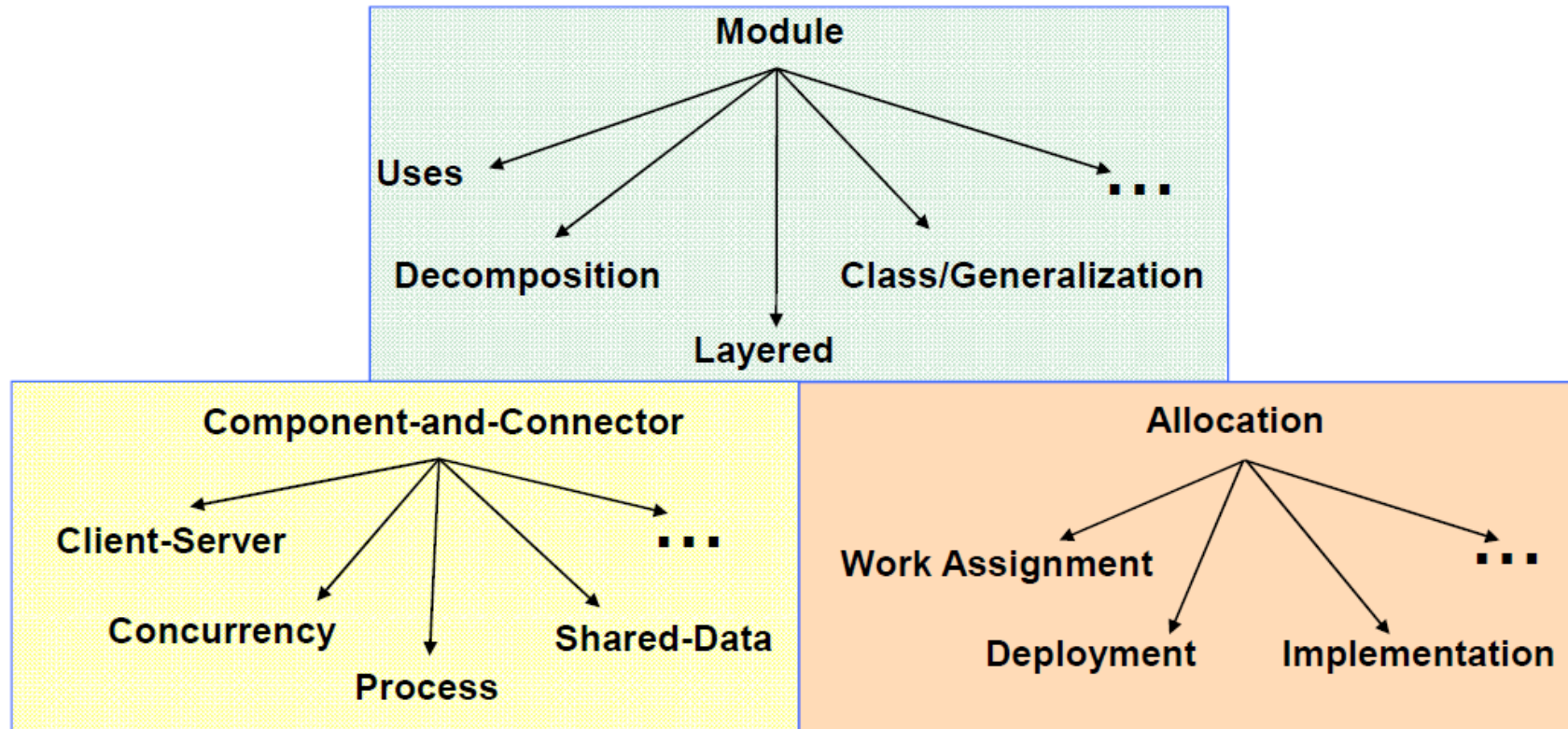
Quiz

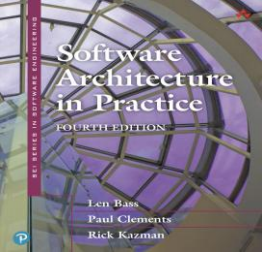
- Architectural structures for software systems can be divided into what three types of structures?
- a) Component, connector, and deployment structure
- b) Control flow, process, and allocation structure
- c) Data flow, component, and communication structure
- d) Allocation, module, and component-and-connectore structure?

Quiz

- Architectural structures for software systems can be divided into what three types of structures?
- a) Component, connector, and deployment structure
- b) Control flow, process, and allocation structure
- c) Data flow, component, and communication structure
- d) Allocation, module, and component-and-connectore structure?

Structure

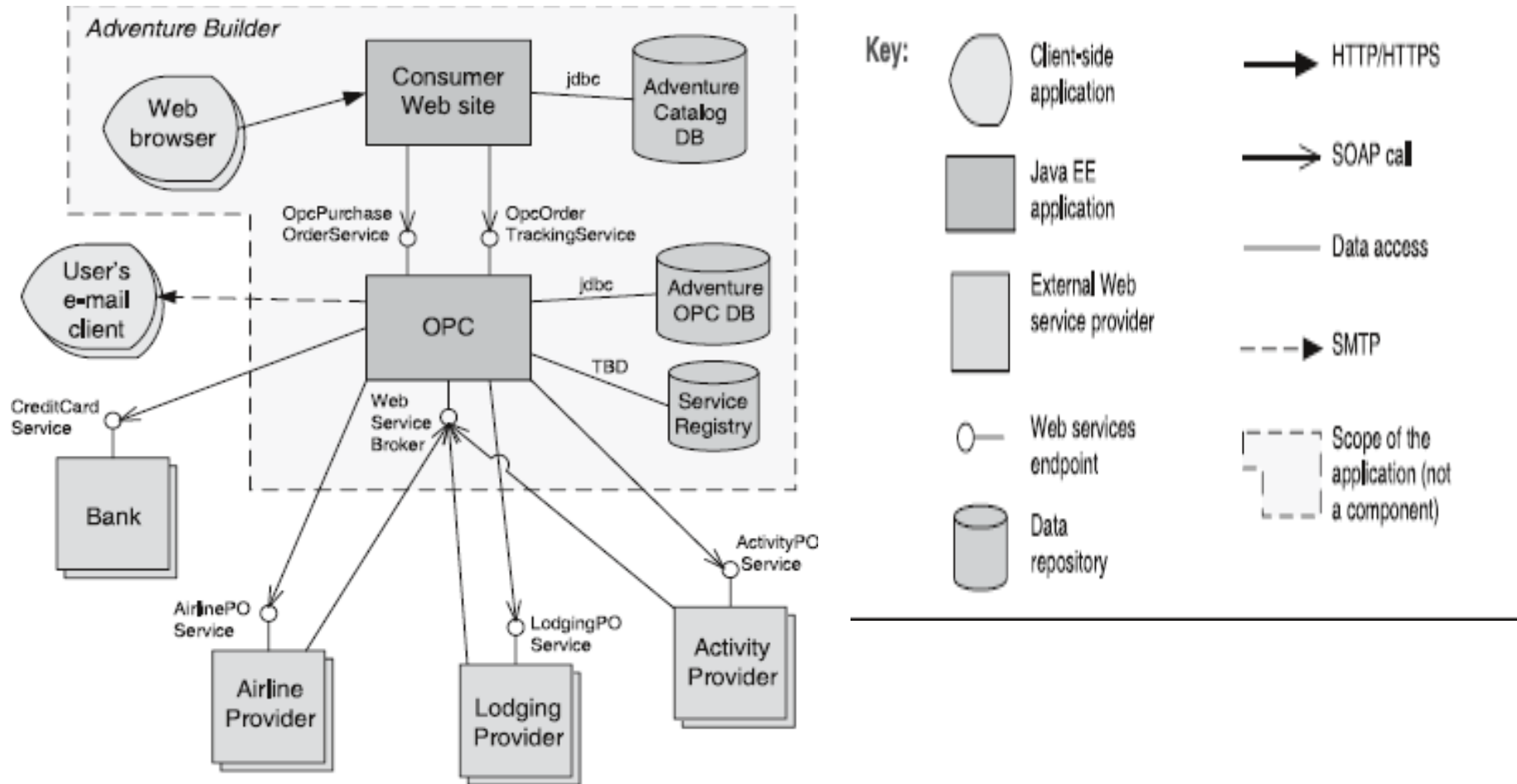




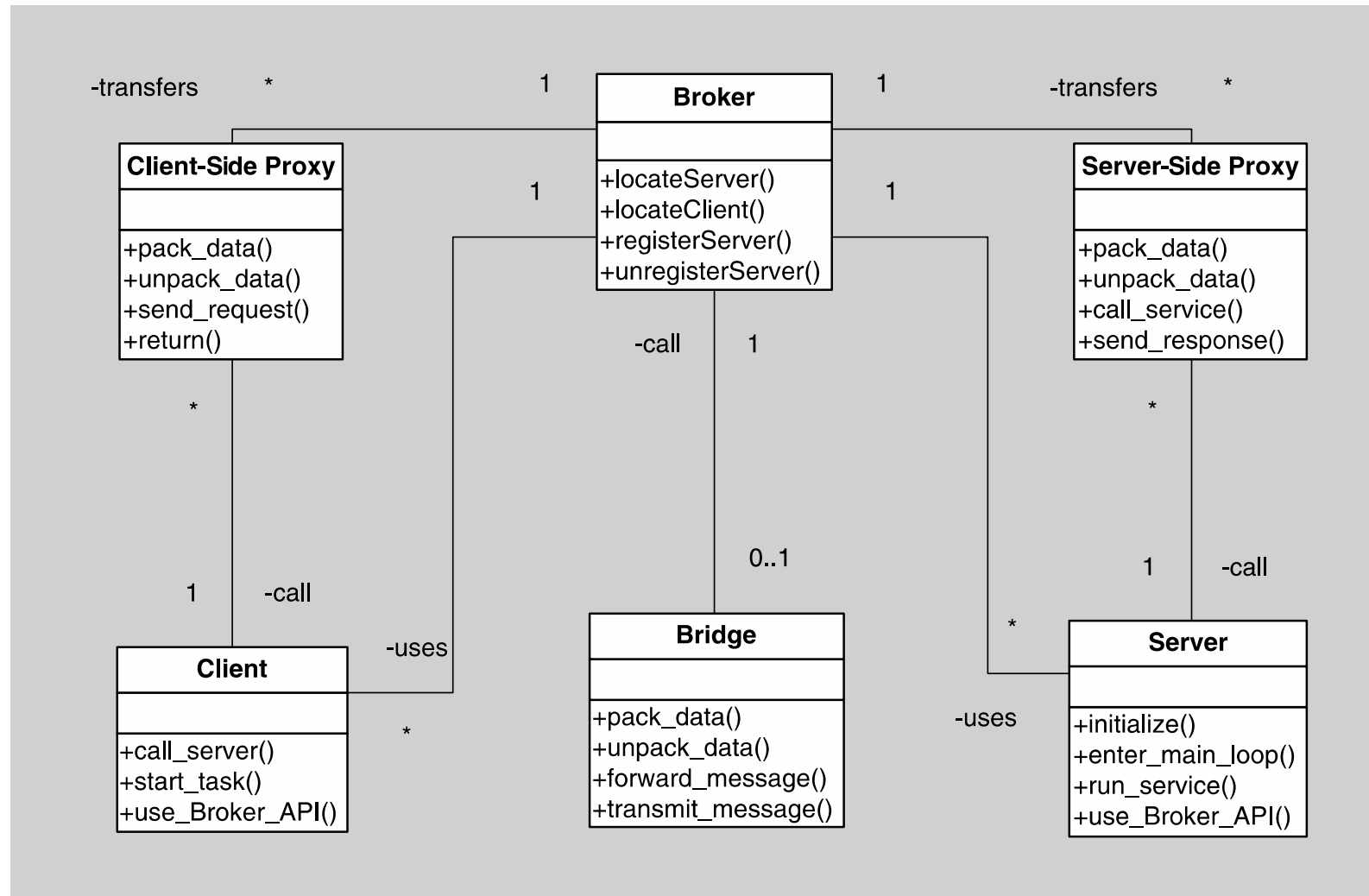
Module Structures

- Module structures embody decisions as to how the system is to be structured as a set of code or data units that have to be constructed or procured.
- In any module structure, the elements are modules of some kind (perhaps classes, or layers, or merely divisions of functionality, all of which are units of implementation).
- Modules are assigned areas of functional responsibility; there is less emphasis in these structures on how the software manifests at runtime.
- Module structures allow us to answer questions such as these:
 - What is the primary functional responsibility assigned to each module?
 - What other software elements is a module allowed to use?
 - What other software does it actually use and depend on?
 - What modules are related to other modules by generalization or specialization (i.e., inheritance) relationships?

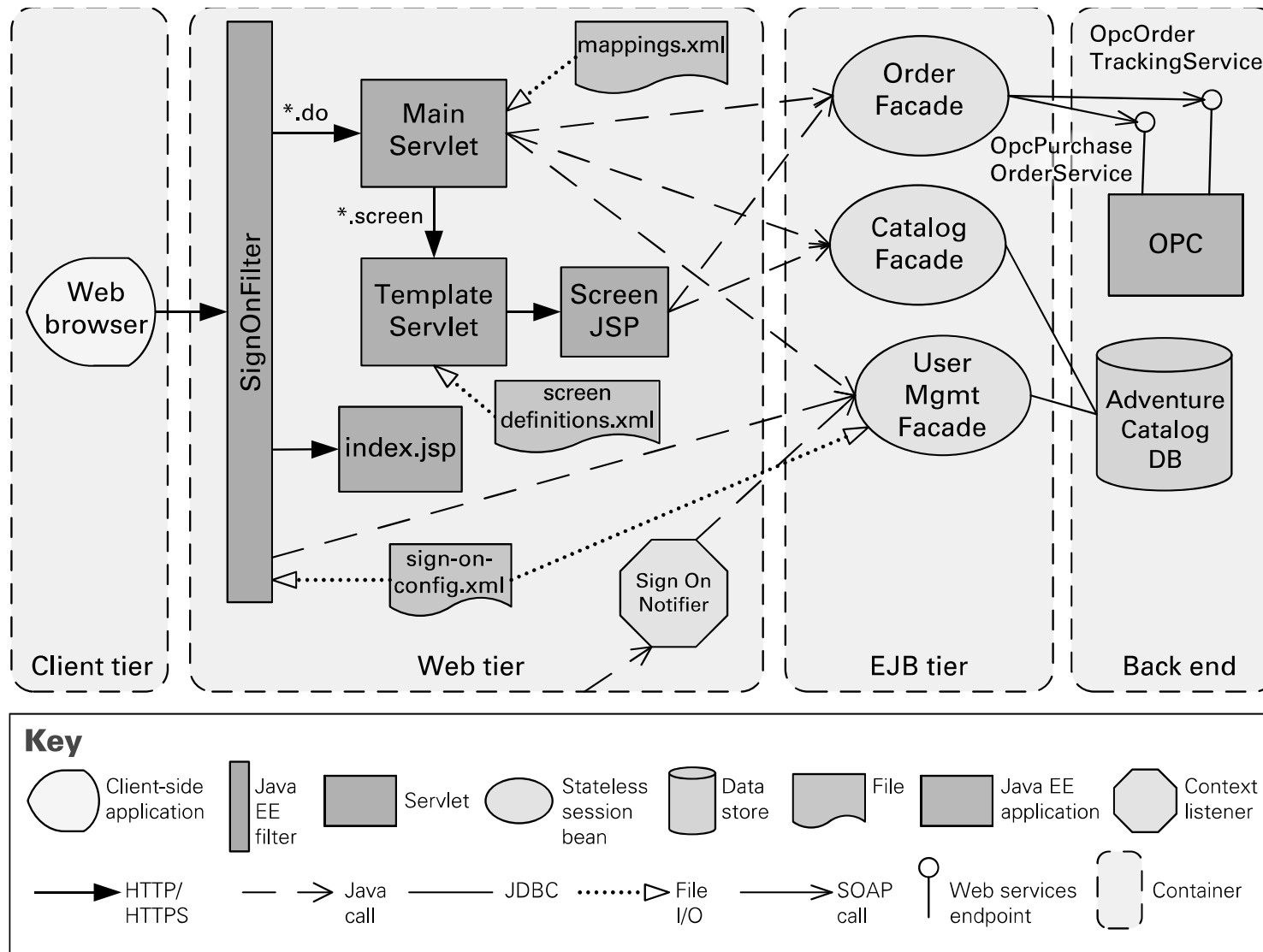
Module Structure Example



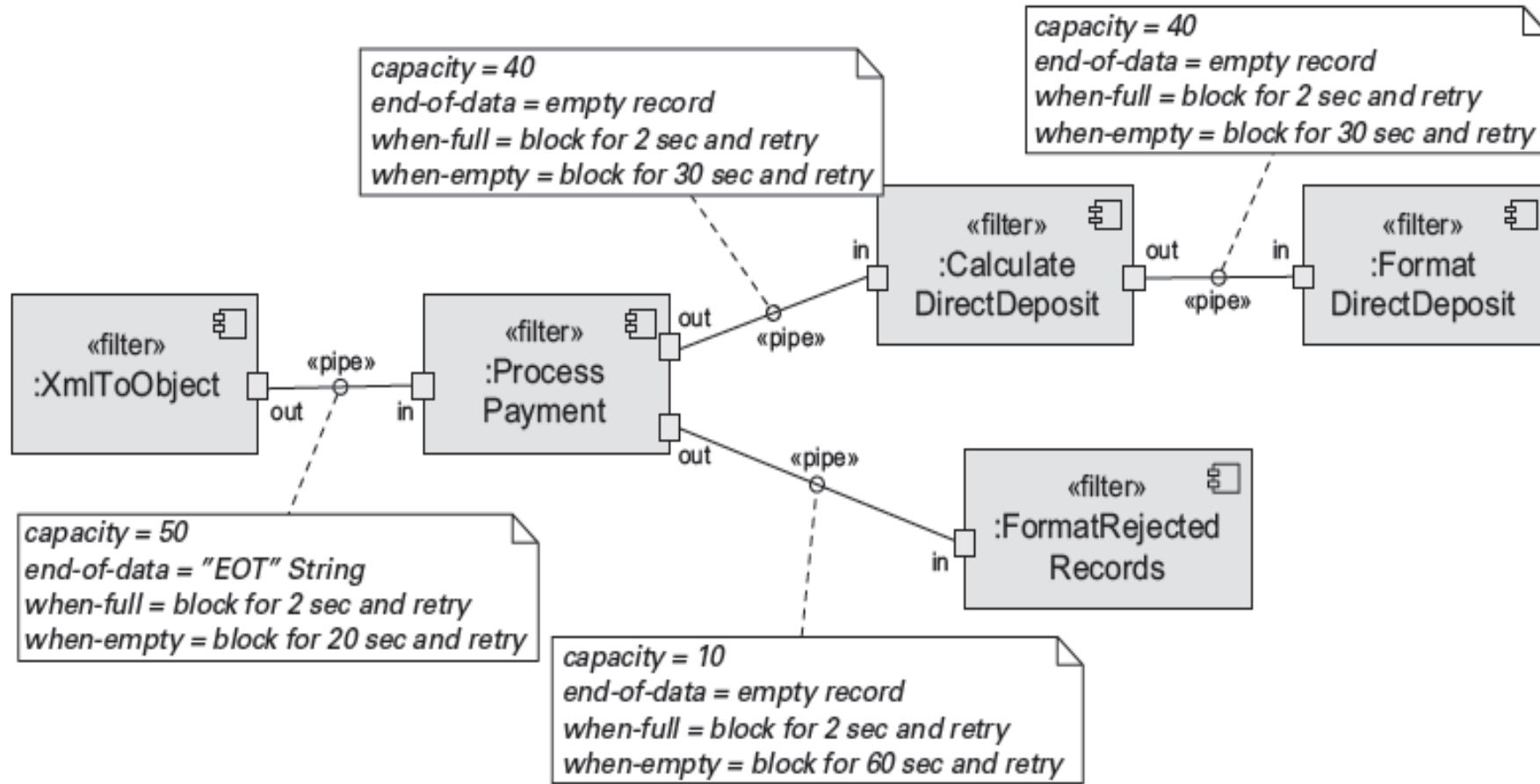
Broker Example

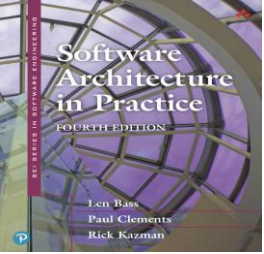


Multi-Tier Example



Pipe and Filter Example

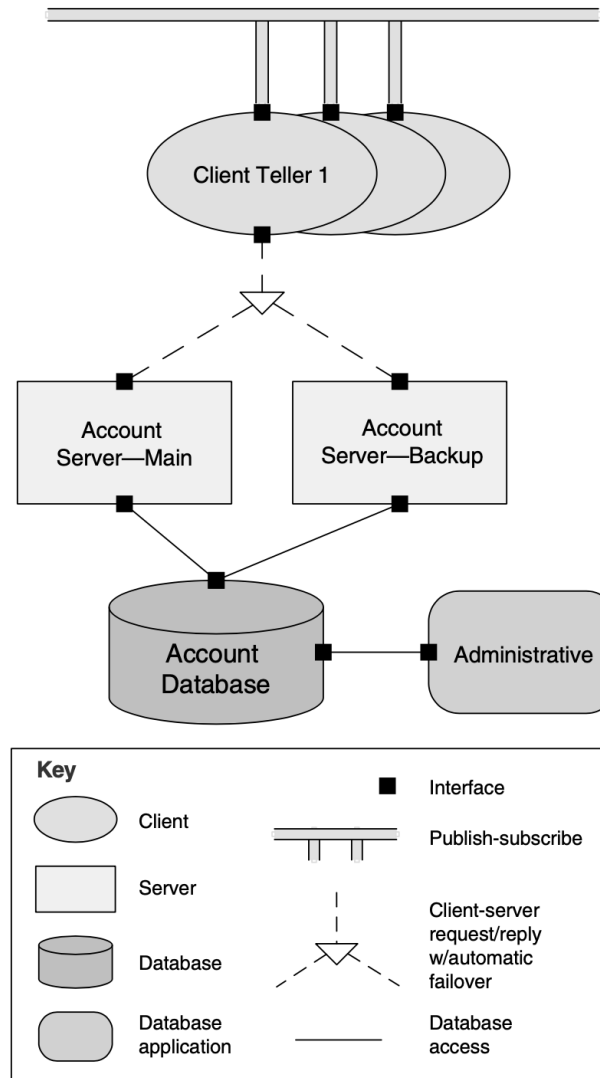


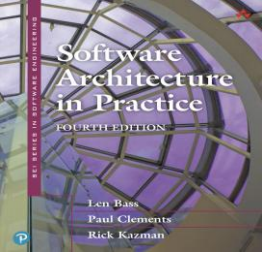


Component-and-connector Structures

- Component-and-connector structures embody decisions as to how the system is to be structured as a set of elements that have runtime behavior (components) and interactions (connectors).
- Elements are runtime components such as services, peers, clients, servers, filters, or many other types of runtime element)
- Connectors are the communication vehicles among components, such as call-return, process synchronization operators, pipes, or others.
- Component-and-connector views help us answer questions such as these:
 - What are the major executing components and how do they interact at runtime?
 - What are the major shared data stores?
 - Which parts of the system are replicated?
 - How does data progress through the system?
 - What parts of the system can run in parallel?
 - Can the system's structure change as it executes and, if so, how?
- Component-and-connector views are crucially important for asking questions about the system's runtime properties such as performance, security, availability, and more.

An Example C&C Structure

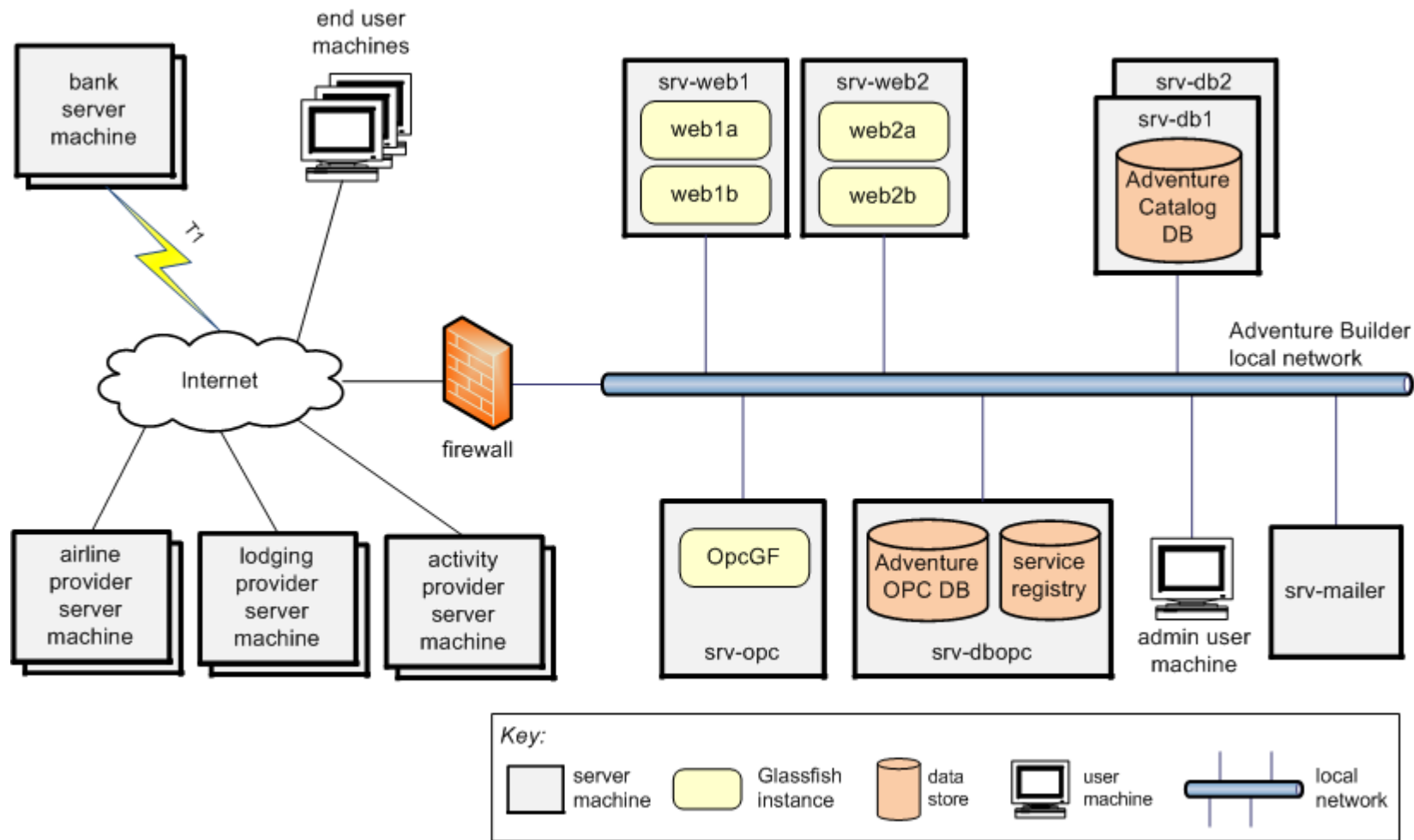


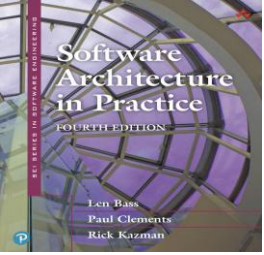


Allocation structures

- Allocation structures show the relationship between the software elements and elements in one or more external environments in which the software is created and executed.
- Allocation views help us answer questions such as these:
 - What processor does each software element execute on?
 - In what directories or files is each element stored during development, testing, and system building?
 - What is the assignment of each software element to development teams?

Allocation Example

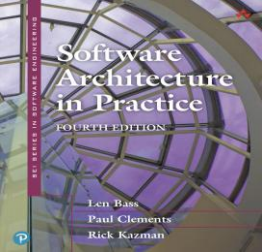




Some Useful Module Structures

Decomposition structure

- The units are modules that are related to each other by the *is-a-submodule-of* relation.
- It shows how modules are decomposed into smaller modules recursively until the modules are small enough to be easily understood.
- Modules often have products (such as interface specifications, code, test plans, etc.) associated with them.
- The decomposition structure determines, to a large degree, the system's modifiability, by assuring that likely changes are localized.
- This structure is often used as the basis for the development project's organization, including the structure of the documentation, and the project's integration and test plans.
- The units in this structure tend to have names that are organization-specific such as "segment" or "subsystem."

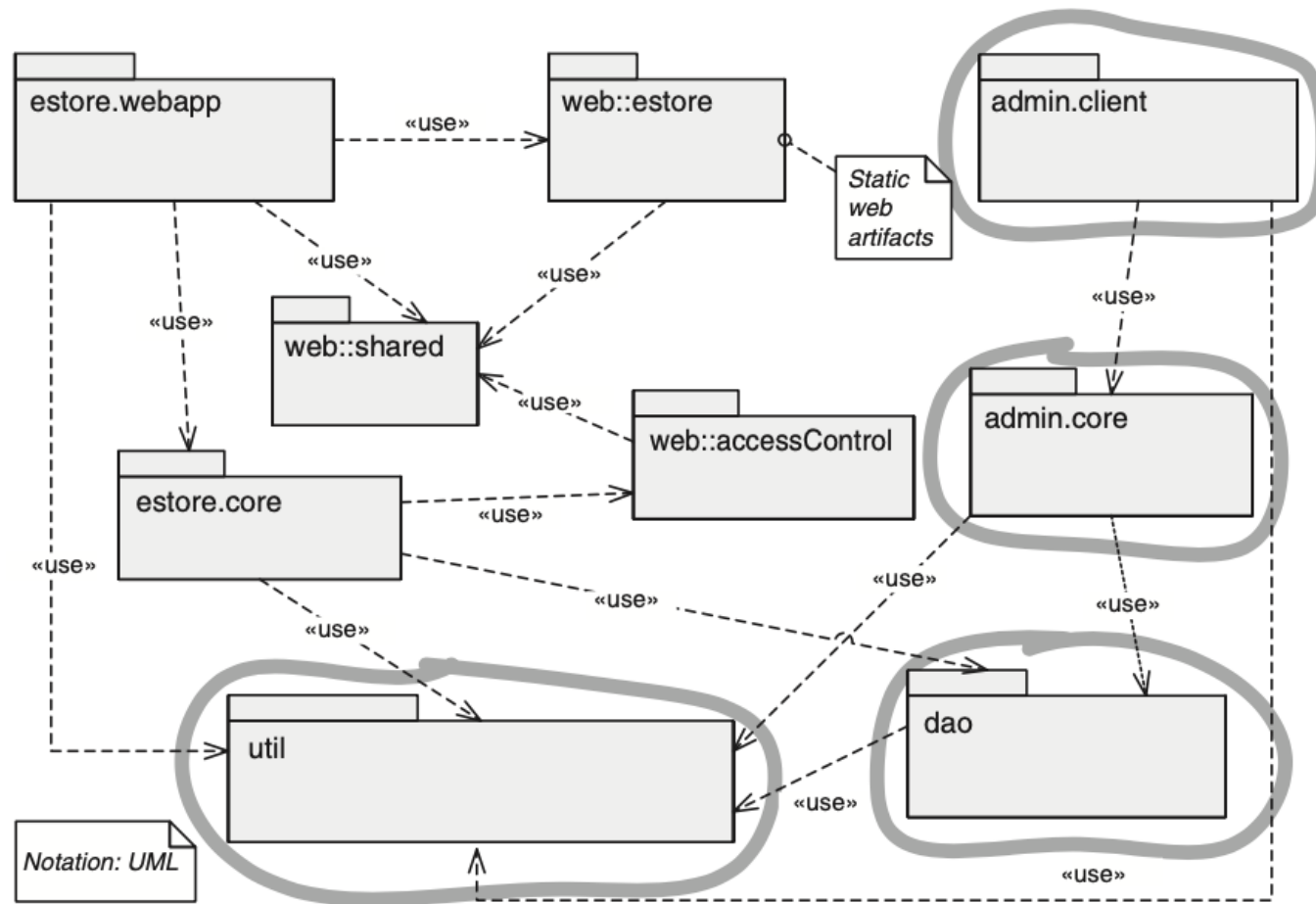


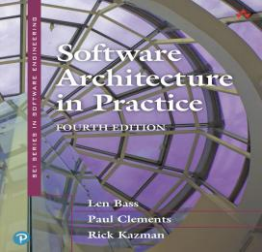
Some Useful Module Structures

Uses structure

- The units here are also modules, perhaps classes.
- The units are related by the *uses* relation, a specialized form of dependency.
- A unit of software *uses* another if the correctness of the first requires the presence of a correctly functioning version (as opposed to a stub) of the second.
- The uses structure is used to engineer systems that can be extended to add functionality, or from which useful functional subsets can be extracted.
- The ability to easily create a subset of a system allows for incremental development.

An Example Uses Structure



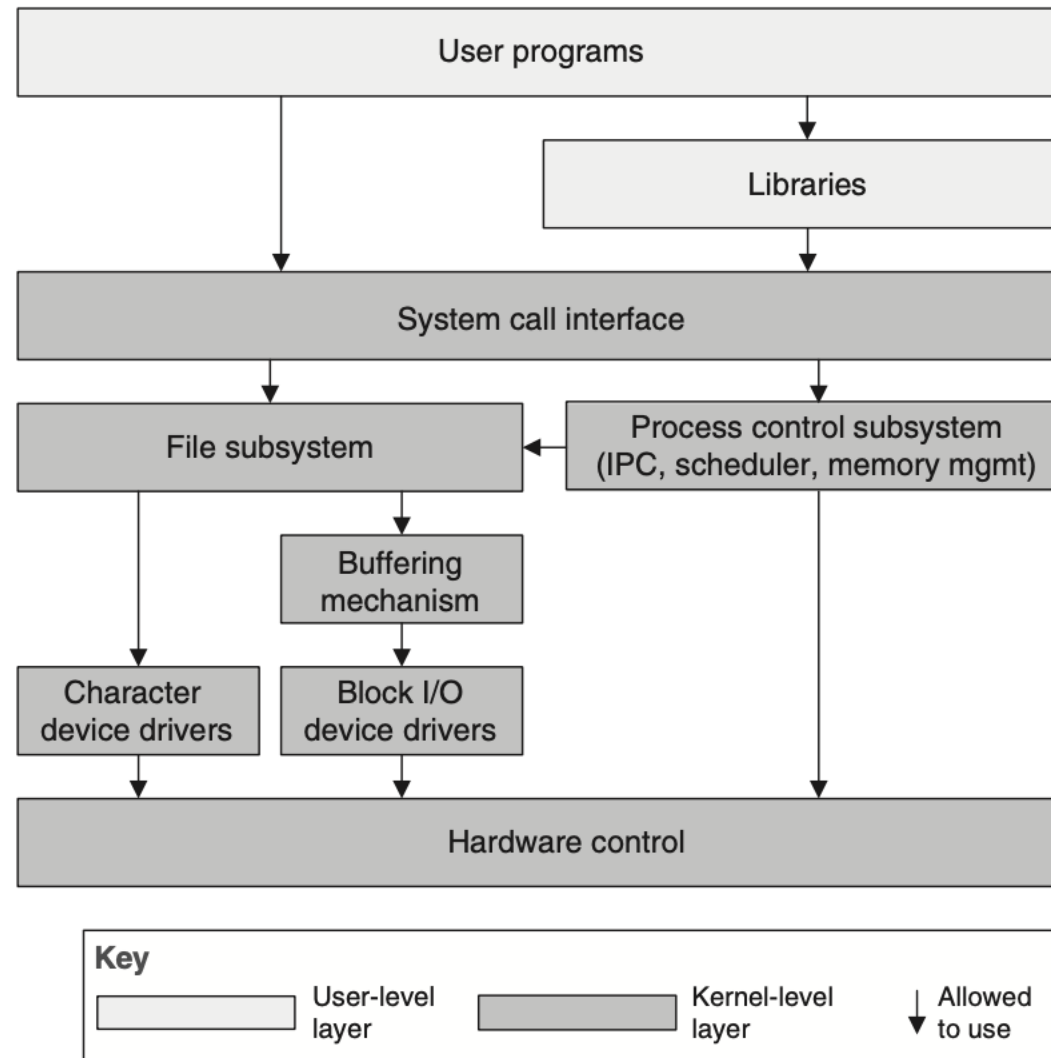


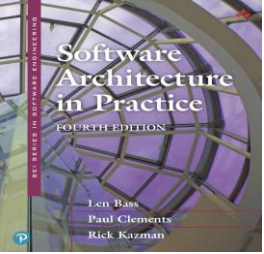
Some Useful Module Structures

Layer structure

- The modules in this structure are called *layers*.
- A layer is an abstract “virtual machine” that provides a cohesive set of services through a managed interface.
- Layers are *allowed to use* other layers in a strictly managed fashion.
 - In strictly layered systems, a layer is only allowed to use a single other layer.
- This structure imbues a system with portability, the ability to change the underlying computing platform.

An Example Layered Structure

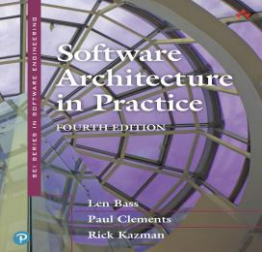




Some Useful Module Structures

Class (or generalization) structure

- The module units in this structure are called *classes*.
- The relation is *inherits from* or *is an instance of*.
- This view supports reasoning about collections of similar behavior or capability
 - e.g., the classes that other classes inherit from and parameterized differences
- The class structure allows one to reason about reuse and the incremental addition of functionality.
- If any documentation exists for a project that has followed an object-oriented analysis and design process, it is typically this structure.

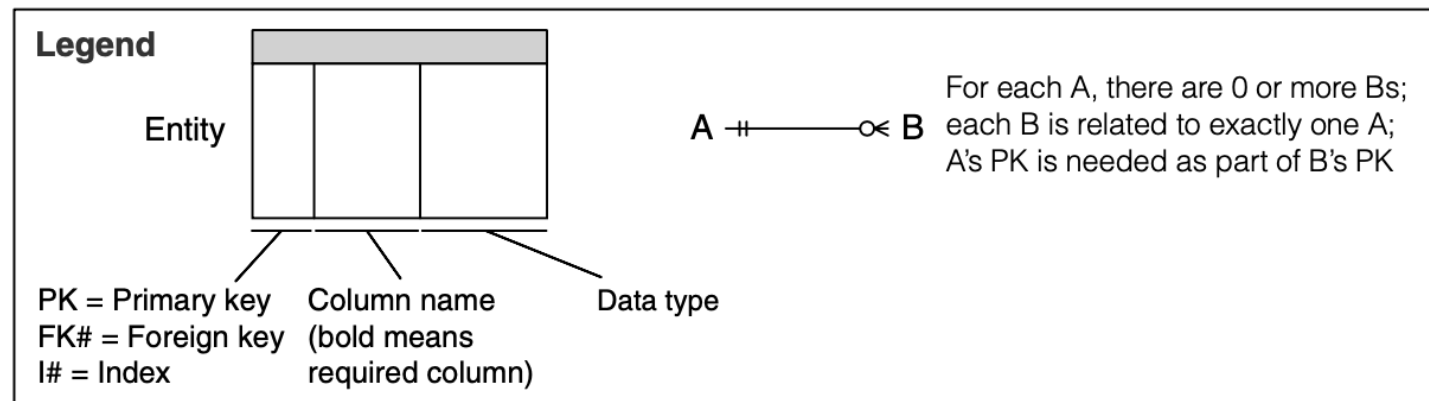
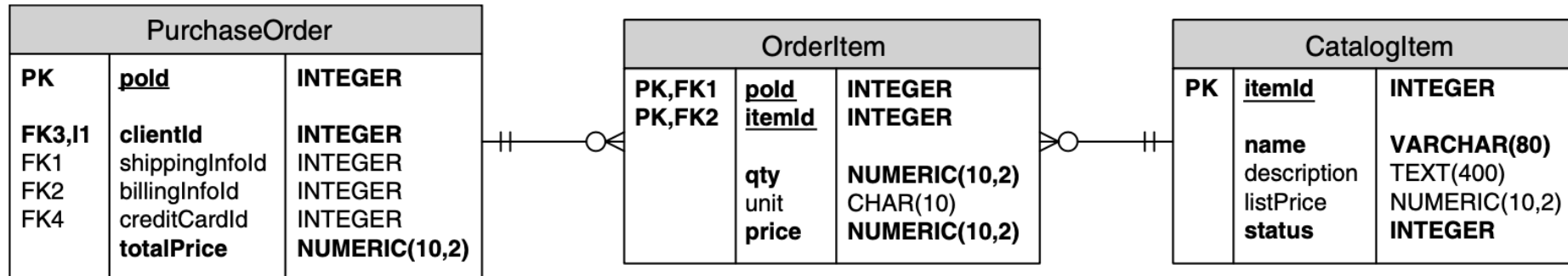


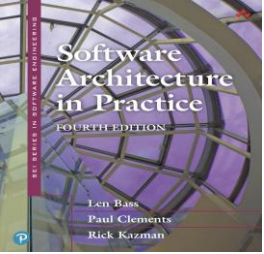
Some Useful Module Structures

Data model

- The data model describes the static information structure in terms of data entities and their relationships.
 - For example, in a banking system, entities will typically include Account, Customer, and Loan.
 - Account has several attributes, such as account number, type (savings or checking), status, and current balance.

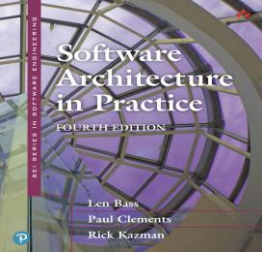
An Example Data Model





Some Useful C&C Structures

- The relation in all component-and-connector structures is attachment, showing how the components and the connectors are hooked together.
- The connectors can be familiar constructs such as “invokes.”
- Useful C&C structures include:
 - Service structure
 - The units are services that interoperate with each other by service coordination mechanisms such as SOAP.
 - The service structure helps to engineer a system composed of components that may have been developed anonymously and independently of each other.
 - Concurrency structure
 - This structure helps determine opportunities for parallelism and the locations where resource contention may occur.
 - The units are components
 - The connectors are their communication mechanisms.
 - The components are arranged into logical threads.

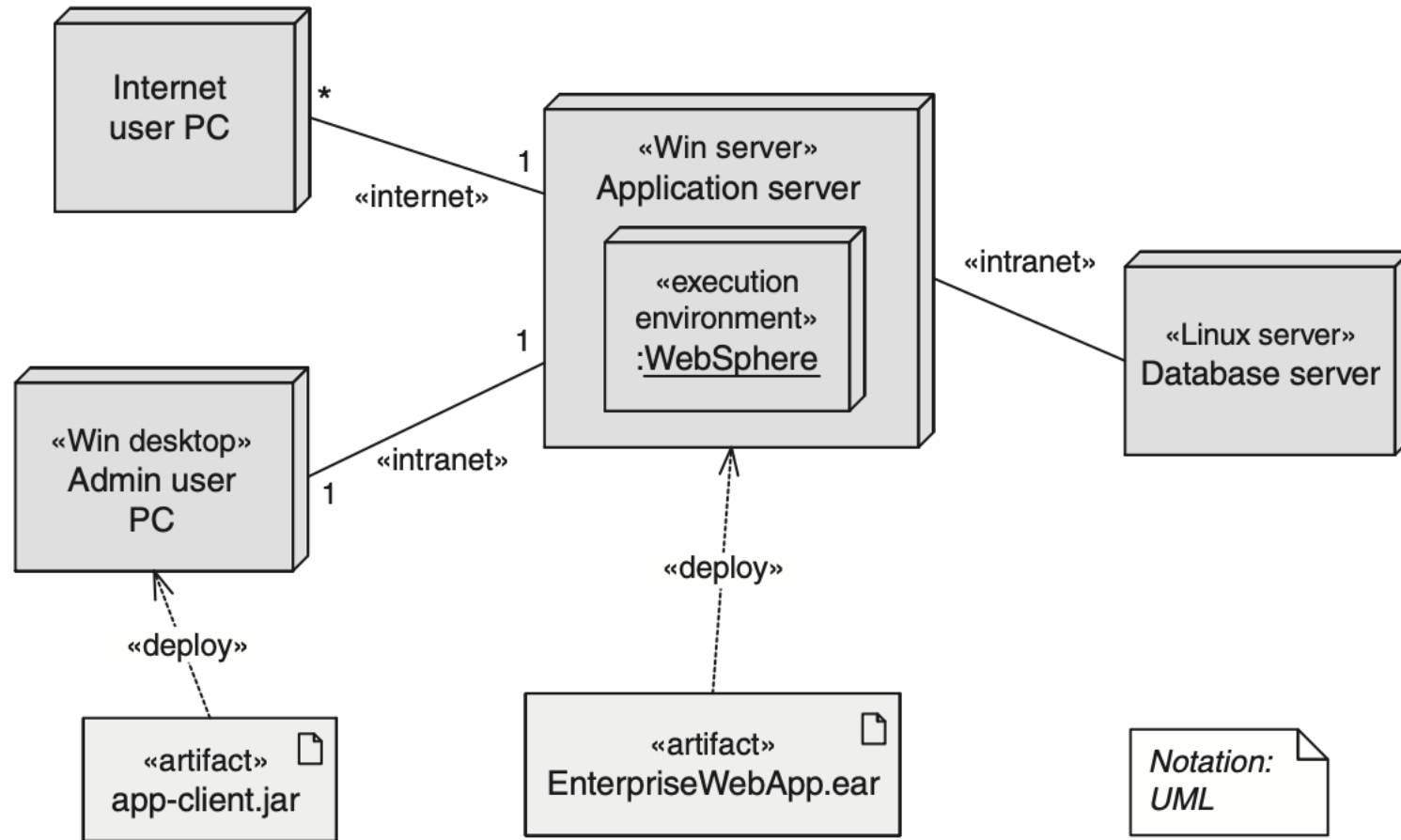


Some Useful Allocation Structures

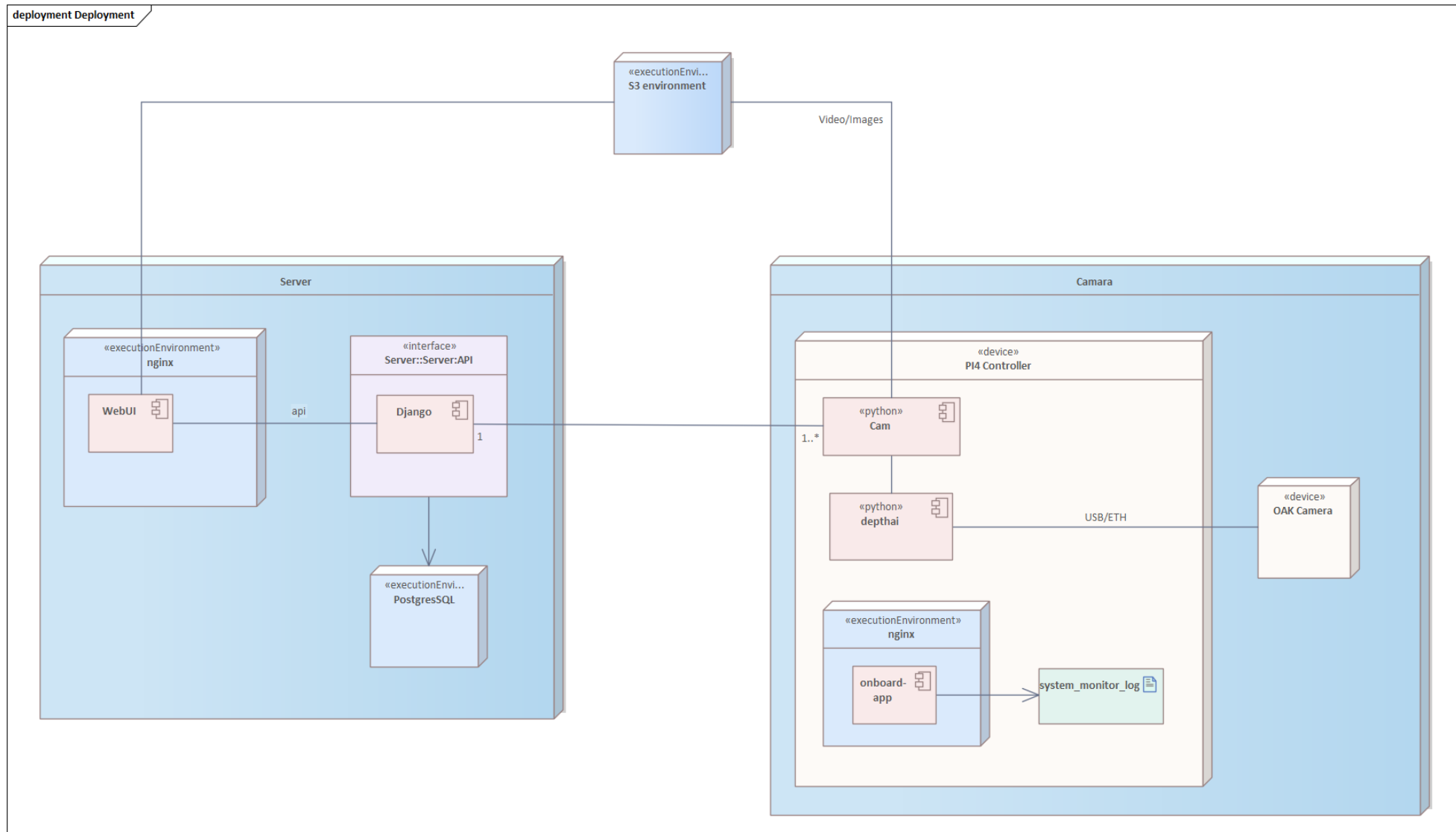
Deployment structure

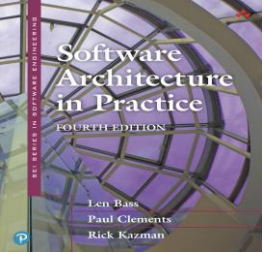
- The deployment structure shows how software is assigned to hardware processing and communication elements.
- The elements are software elements (usually a process from a C&C view), hardware entities (processors), and communication pathways.
- Relations are allocated-to, showing on which physical units the software elements reside, and migrates-to if the allocation is dynamic.
- This structure can be used to reason about performance, data integrity, security, and availability.
- It is of particular interest in distributed and parallel systems.

An Example Deployment Structure



Allocation diagram ex





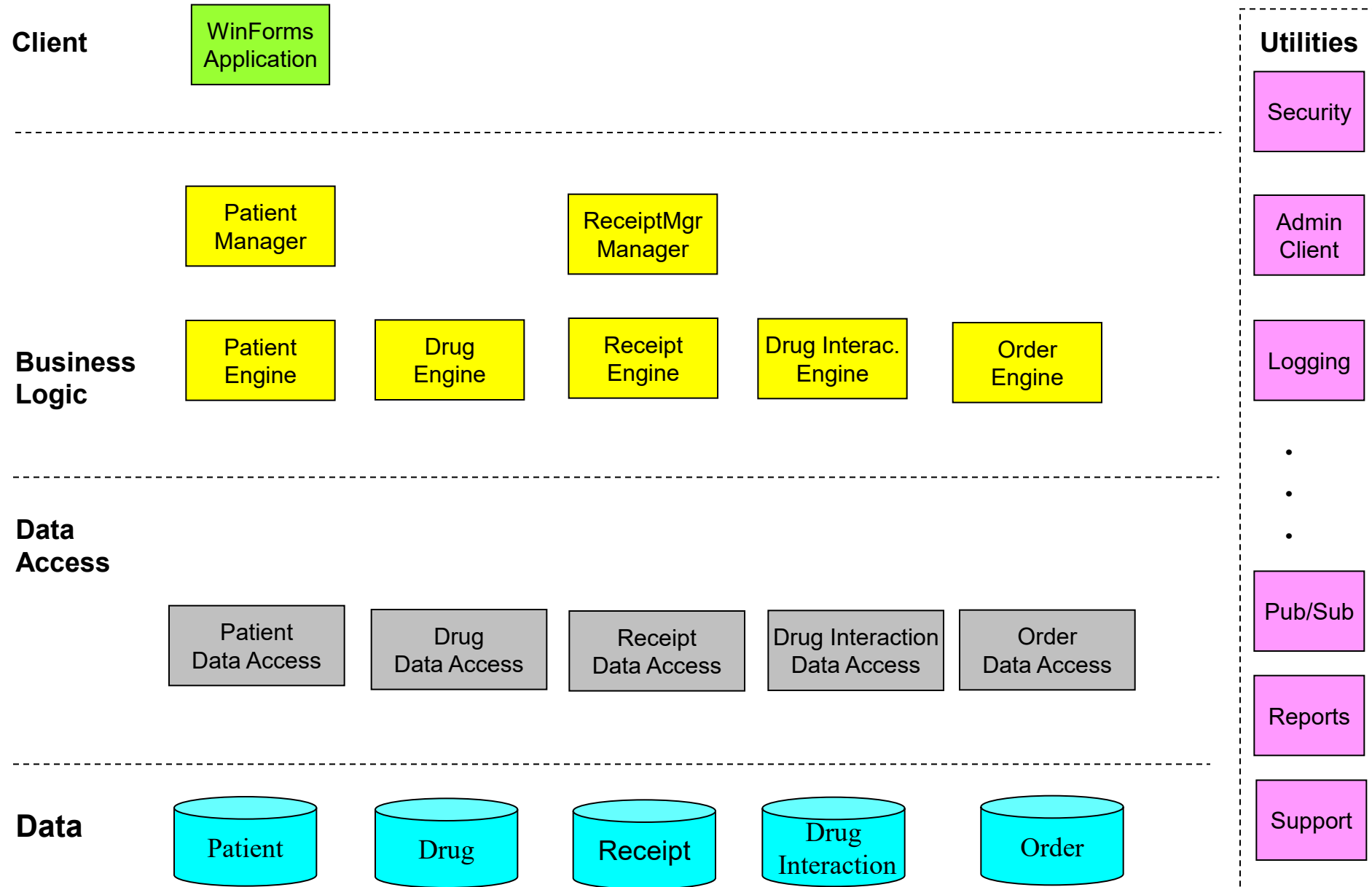
Some Useful Allocation Structures

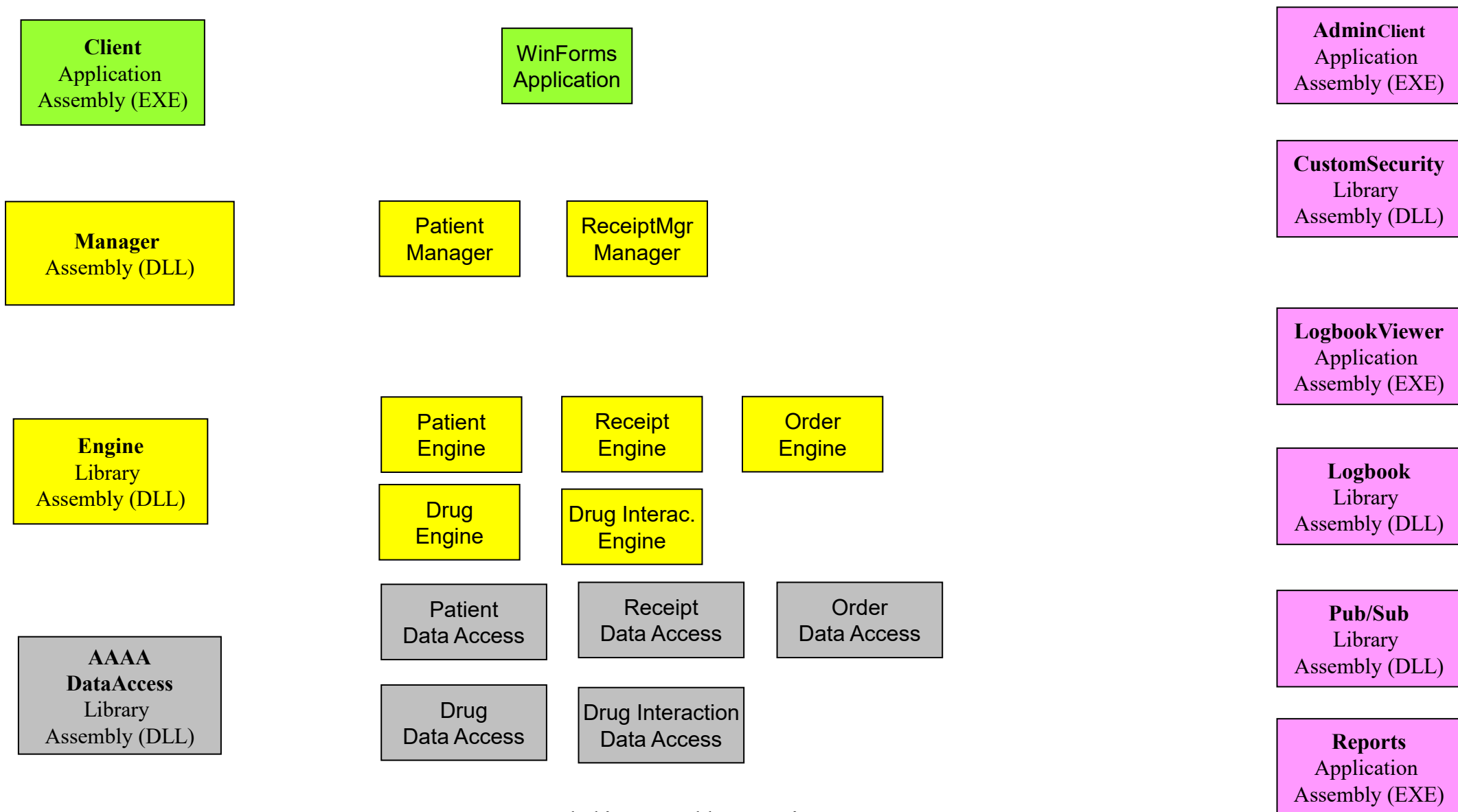
Implementation structure

- This structure shows how software elements (usually modules) are mapped to the file structure(s) in the system's development, integration, or configuration control environments.
- This is critical for the management of development activities and build processes.

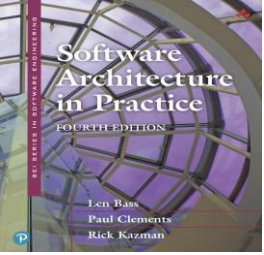
Work assignment structure

- This structure assigns responsibility for implementing and integrating the modules to the teams who will carry it out.
- Having a work assignment structure be part of the architecture makes it clear that the decision about who does the work has architectural as well as management implications.
- The architect will know the expertise required on each team.
- This structure will also determine the major communication pathways among the teams: regular teleconferences, wikis, email lists, and so forth.





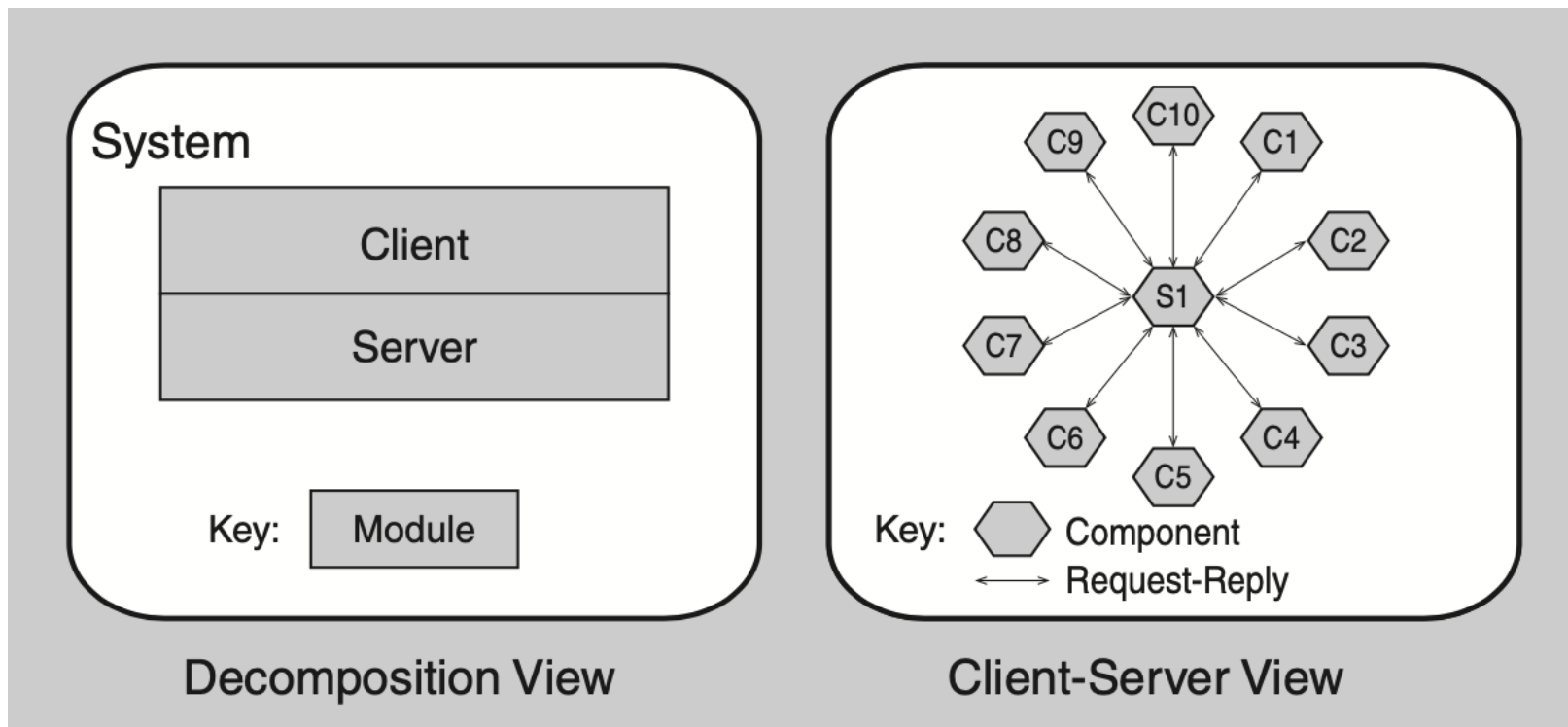
Assemblies Allocation

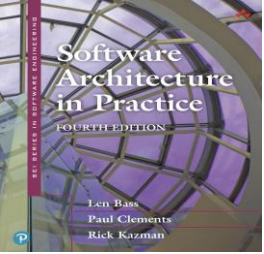


Relating Structures to Each Other

- Elements of one structure will be related to elements of other structures, and we need to reason about these relations.
 - A module in a decomposition structure may be manifested as one, part of one, or several components in one of the component-and-connector structures.
- In general, mappings between structures are many to many.

Two Views of a Client-Server System





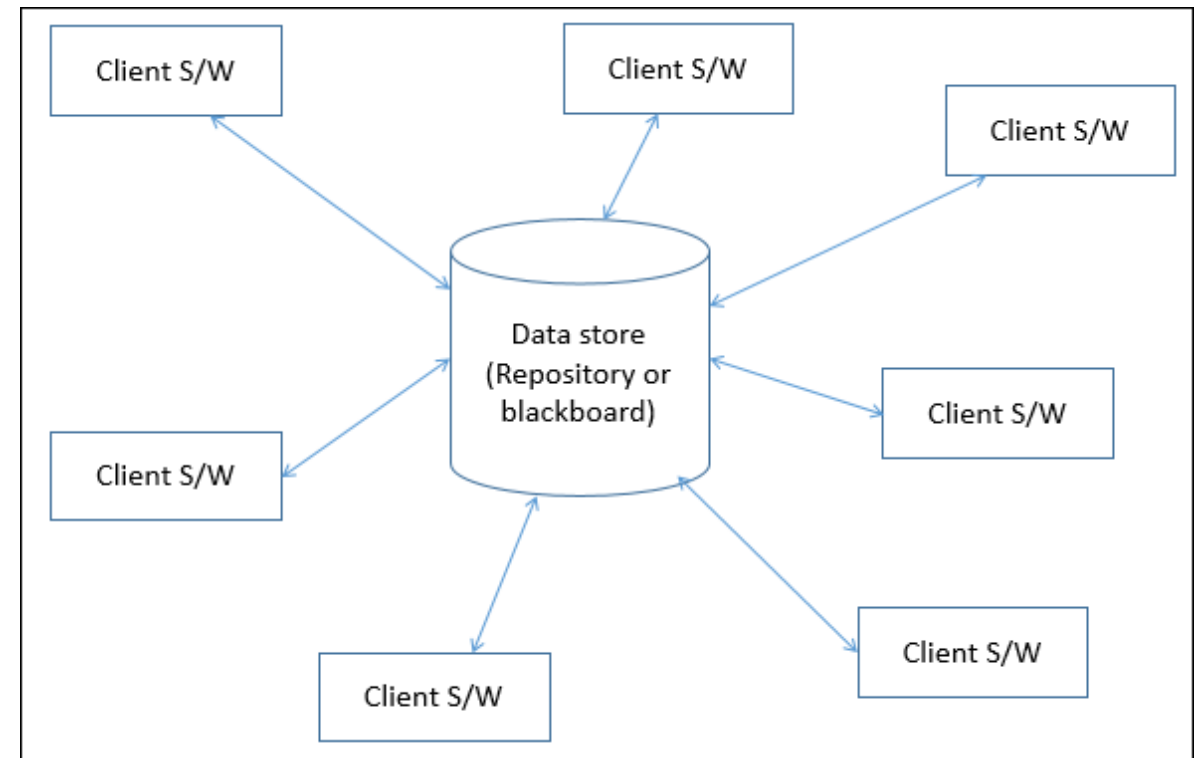
Architectural Patterns

- Architectural elements can be composed in ways that solve particular problems.
 - The compositions have been found useful over time, and over many different domains
 - They have been documented and disseminated.
 - These compositions of architectural elements, called architectural patterns.
 - Patterns provide packaged strategies for solving some of the problems facing a system.
- An architectural pattern delineates the element types and their forms of interaction used in solving the problem.
- We will be visiting many patterns throughout this course.

Architectural Patterns

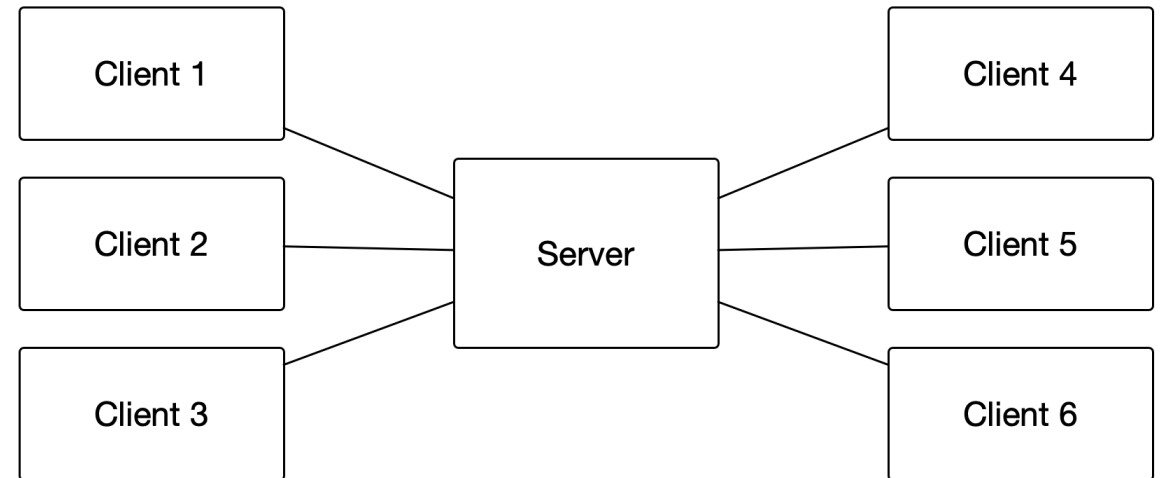
Common component-and-connector type patterns:

- Shared-data (or repository) pattern.
 - This pattern comprises components and connectors that create, store, and access persistent data.
 - The repository usually takes the form of a (commercial) database.
 - The connectors are protocols for managing the data, such as SQL.



Architectural Patterns

- Client-server pattern.
 - The components are the clients and the servers.
 - The connectors are protocols and messages they share among each other to carry out the system's work.



Architectural Patterns

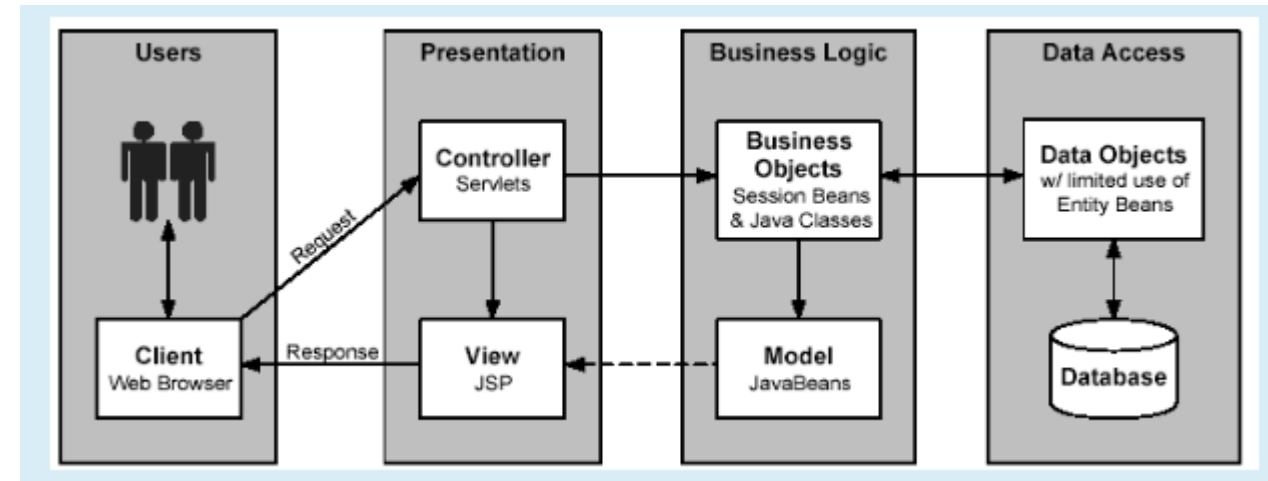
Common allocation patterns:

- Multi-tier pattern
 - Describes how to distribute and allocate the components of a system in distinct subsets of hardware and software, connected by some communication medium.
 - This pattern specializes the generic deployment (software-to-hardware allocation) structure.
- Competence center pattern and platform pattern
 - These patterns specialize a software system's work assignment structure.
 - In competence center, work is allocated to sites depending on the technical or domain expertise located at a site.
 - In platform, one site is tasked with developing reusable core assets of a software product line, and other sites develop applications that use the core assets.

Architectural Patterns

Common allocation patterns:

- Multi-tier pattern
 - Describes how to distribute and allocate the components of a system in distinct subsets of hardware and software, connected by some communication medium.
 - This pattern specializes the generic deployment (software-to-hardware allocation) structure.



Summary

- The software architecture of a system is the set of structures needed to reason about the system, which comprise software elements, relations among them, and properties of both.
- A structure is a set of elements and the relations among them.
- A view is a representation of a coherent set of architectural elements.
A view is a representation of one or more structures.

Summary

- There are three categories of structures:
 - Module structures show how a system is to be structured as a set of code or data units that have to be constructed or procured.
 - Component-and-connector structures show how the system is to be structured as a set of elements that have runtime behavior (components) and interactions (connectors).
 - Allocation structures show how the system will relate to nonsoftware structures in its environment (such as CPUs, file systems, networks, development teams, etc.).
- Structures represent the primary engineering leverage points of an architecture.
- Every system has a software architecture, but this architecture may be documented and disseminated, or it may not be.
- There is no such thing as an inherently good or bad architecture. Architectures are either more or less fit for some purpose.